Semantic Segmentation and Dense Image Labeling

Many slides from Lana Lazebnik

Visual Recognition

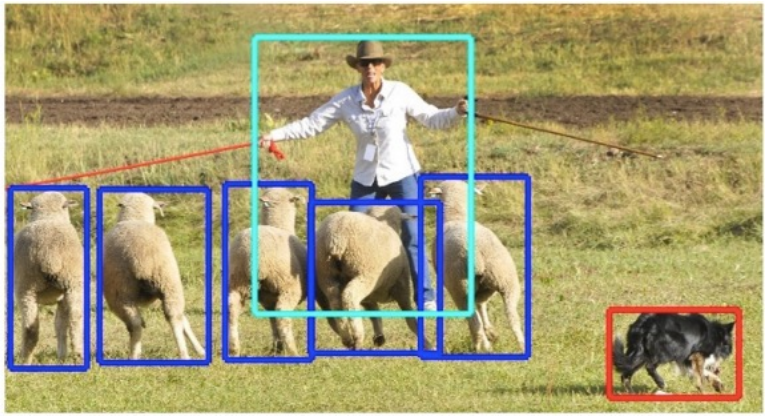


image classification

Visual Recognition



image classification



object detection

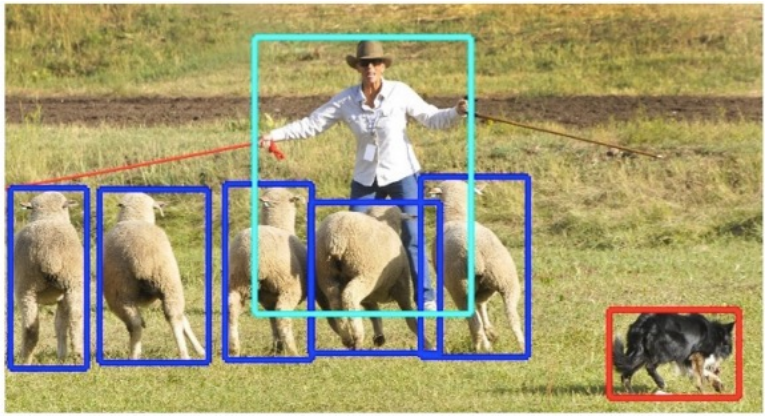


instance segmentation

Visual Recognition



image classification



object detection



semantic segmentation



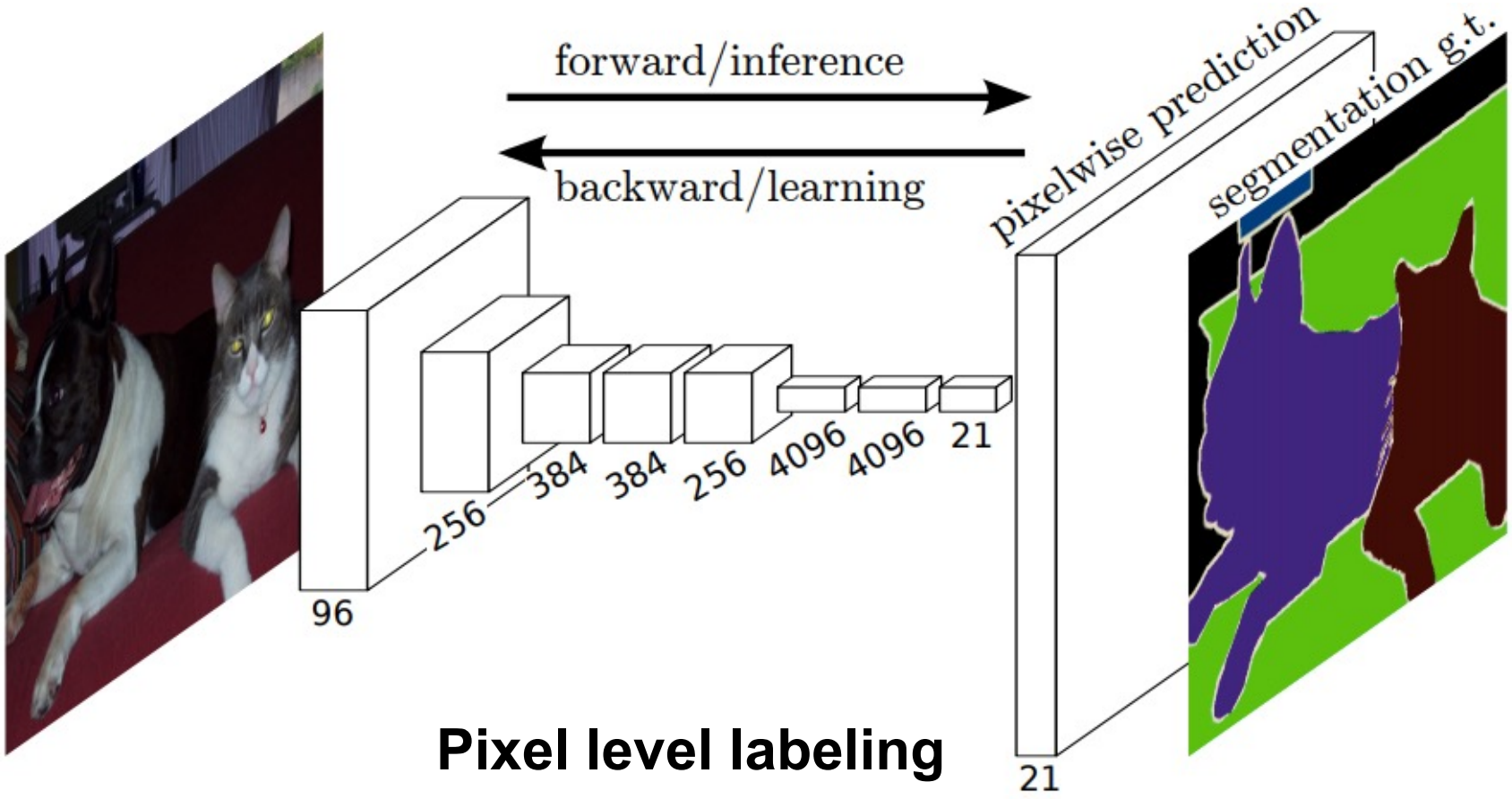
instance segmentation

Semantic Segmentation



PSPNet

Semantic Segmentation as Dense Prediction

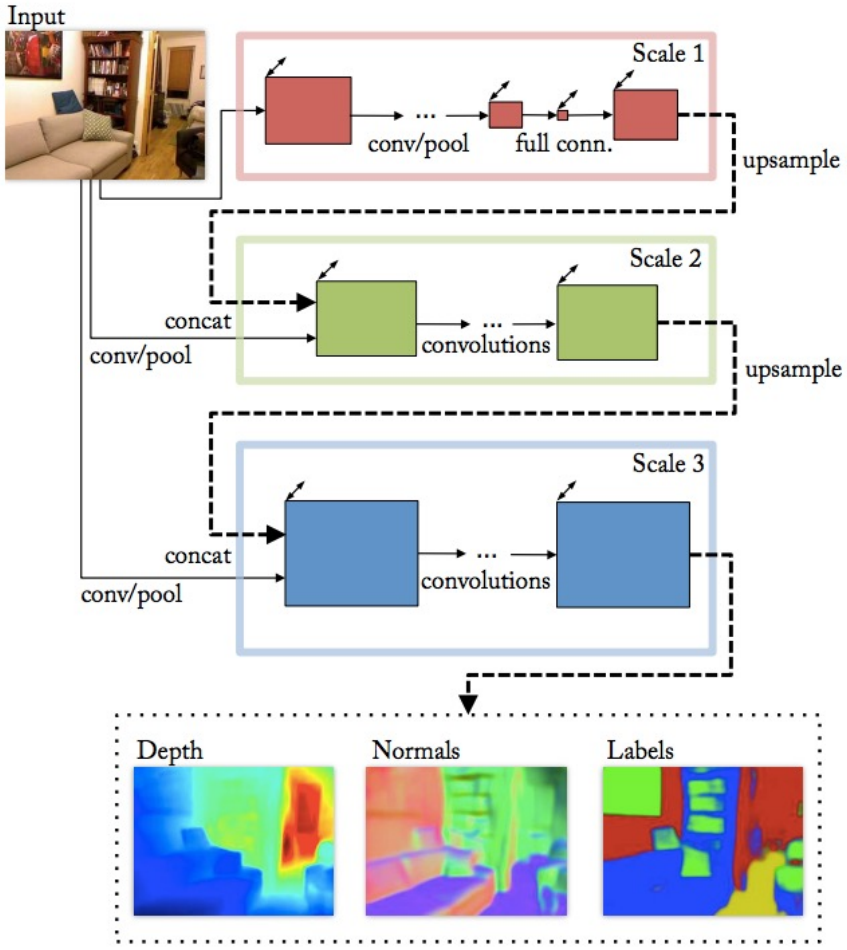


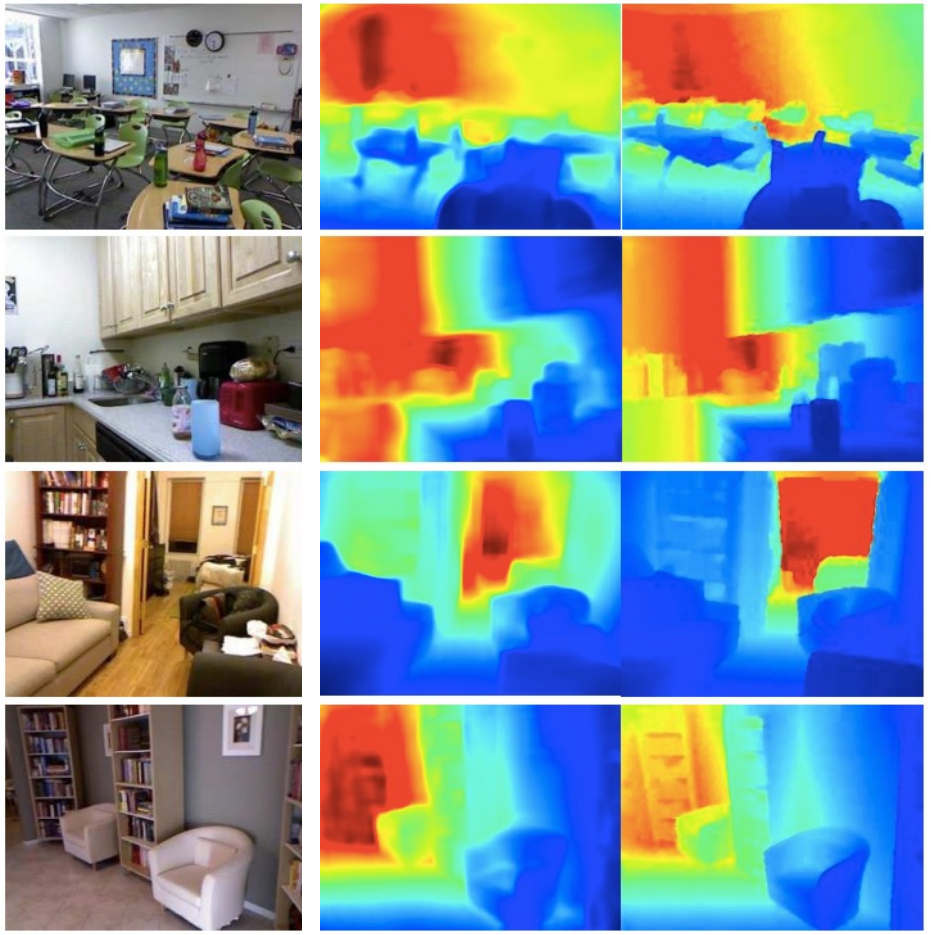
J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

Other dense prediction tasks

- Depth estimation
- Surface normal estimation
- Colorization
- Edge detection
- ….

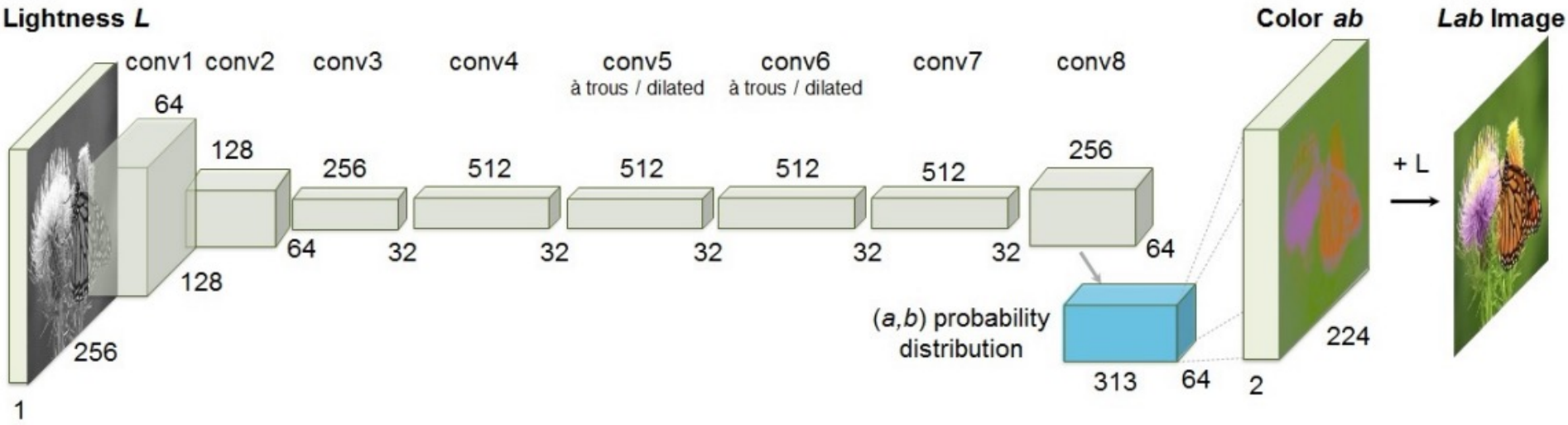
Depth Estimation

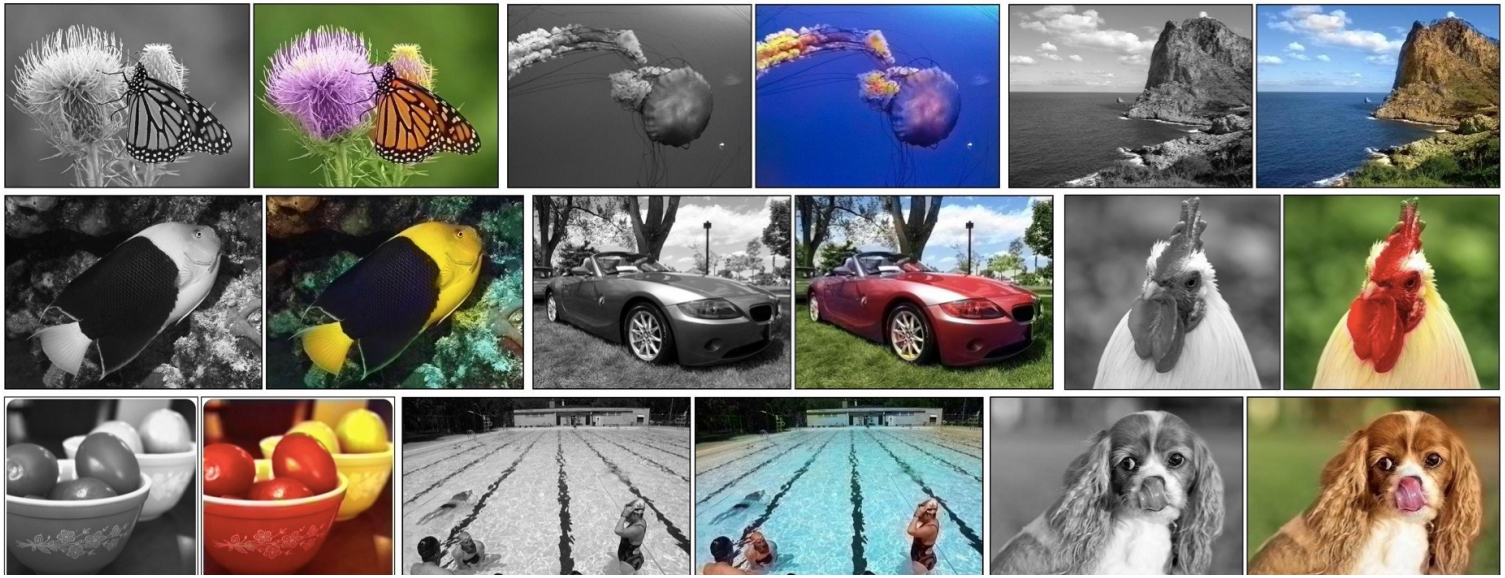




D. Eigen and R. Fergus, Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture, ICCV 2015

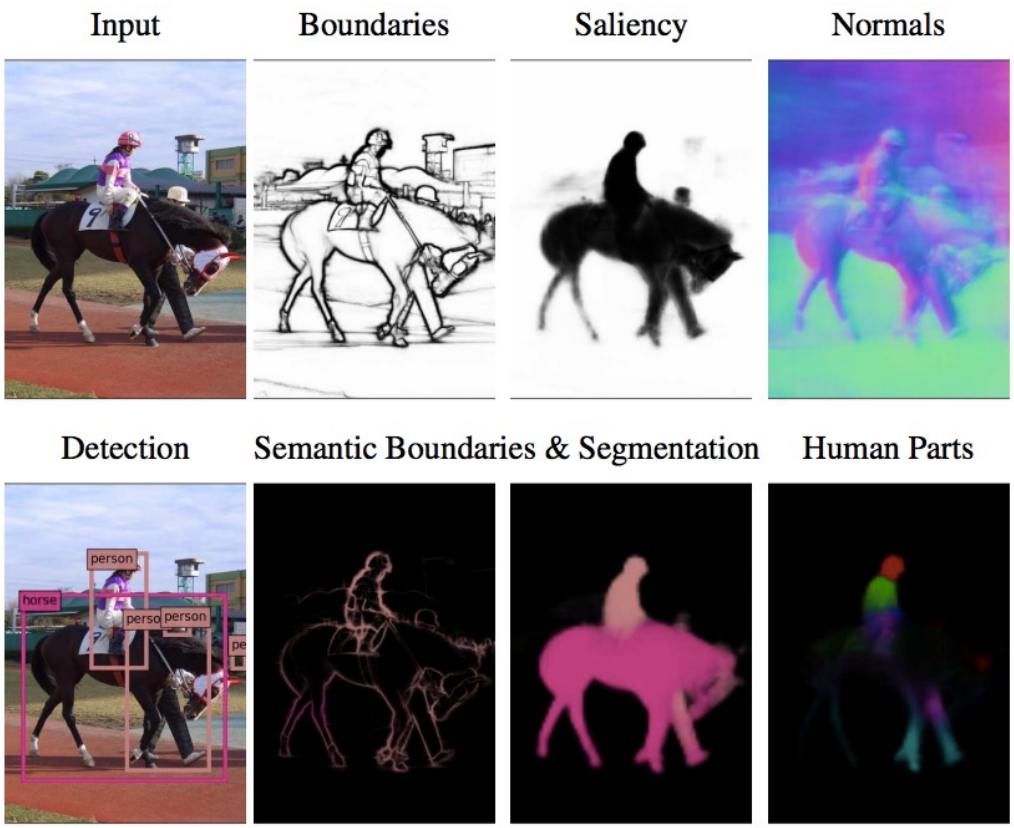
Colorization





R. Zhang, P. Isola, and A. Efros, Colorful Image Colorization, ECCV 2016

Multi-task Dense Labeling



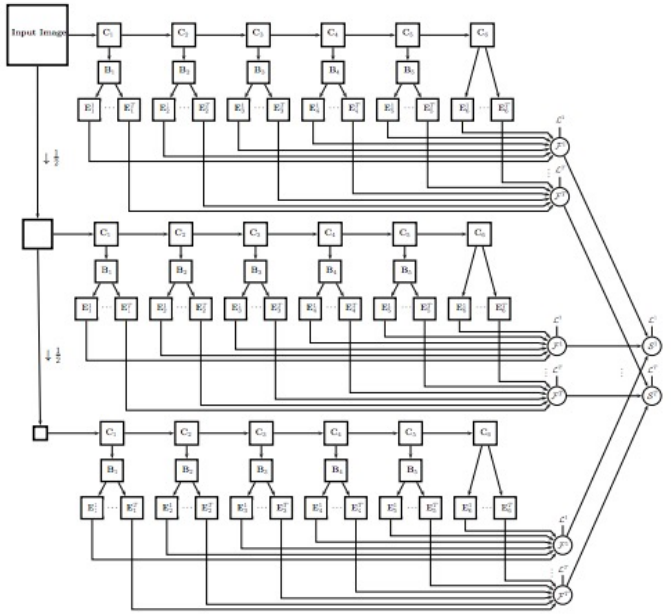
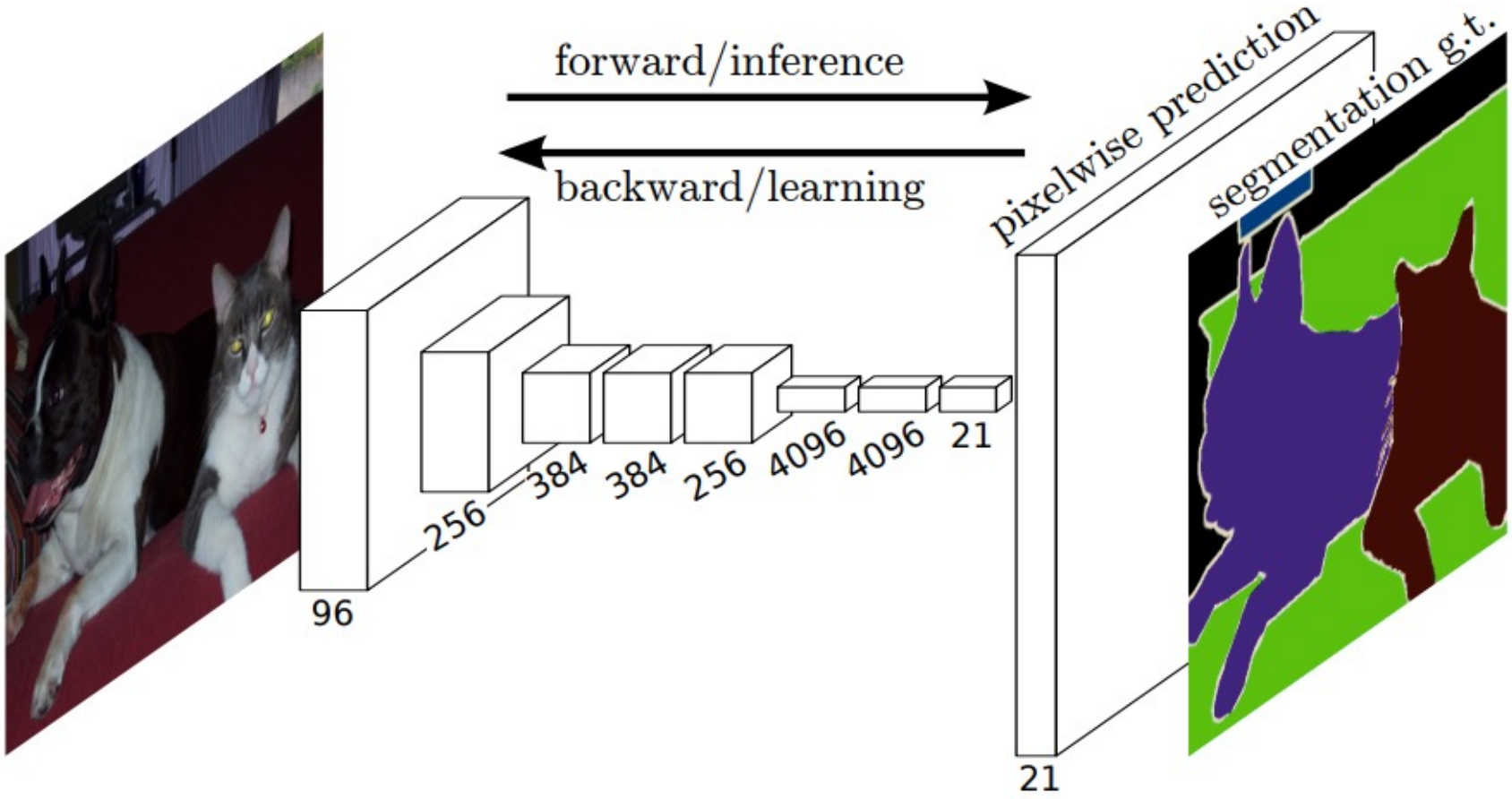


Figure 2: UberNet architecture: an image pyramid is formed by successive down-sampling operations, and each image is processed by a CNN with tied weights; the responses of the network at consecutive layers ($\mathbf{C}_i$) are processed with Batch Normalization ($\mathbf{B}_i$) and then fed to task-specific skip layers ($\mathbf{E}_i^t$); these are combined across network layers ($\mathcal{F}^t$) and resolutions ($\mathcal{S}^t$) and trained using task-specific loss functions ($\mathcal{L}^t$), while the whole architecture is jointly trained end-to-end. For simplicity we omit the interpolation and detection layers mentioned in the text.

I. Kokkinos, UberNet: Training a Universal Convolutional Neural Network for Low-, Mid-, and High-Level Vision using Diverse Datasets and Limited Memory, ICCV 2017

Semantic Segmentation



J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

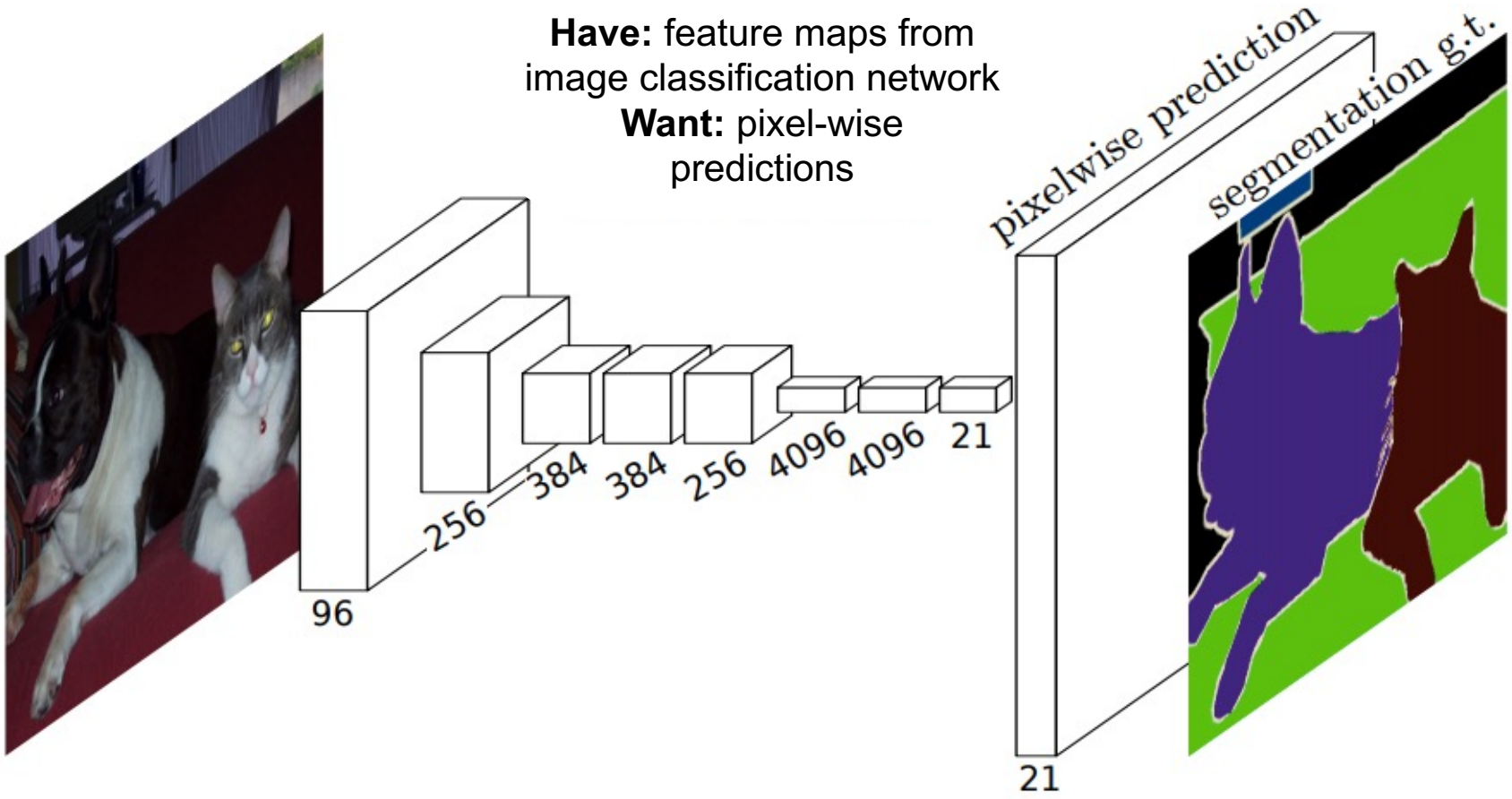
Semantic Segmentation: Evaluation

- **Metric: mean IoU**
  - Pixel accuracy is not a good indicator (bias towards the major categories, e.g., background)
  - Intersection over union (IoU) of predicted and ground truth masks for each class
  - IoU is further averaged over classes (mIoU)
  - Not much about the instances / boundaries …

class	mean	bg																				
acc	69.6	91.9	85.6	37.3	83.2	62.5	66	85.1	80.7	84.9	27.2	73.3	57.5	78.1	79.2	81.1	77.1	53.6	74	49.2	71.7	63.3

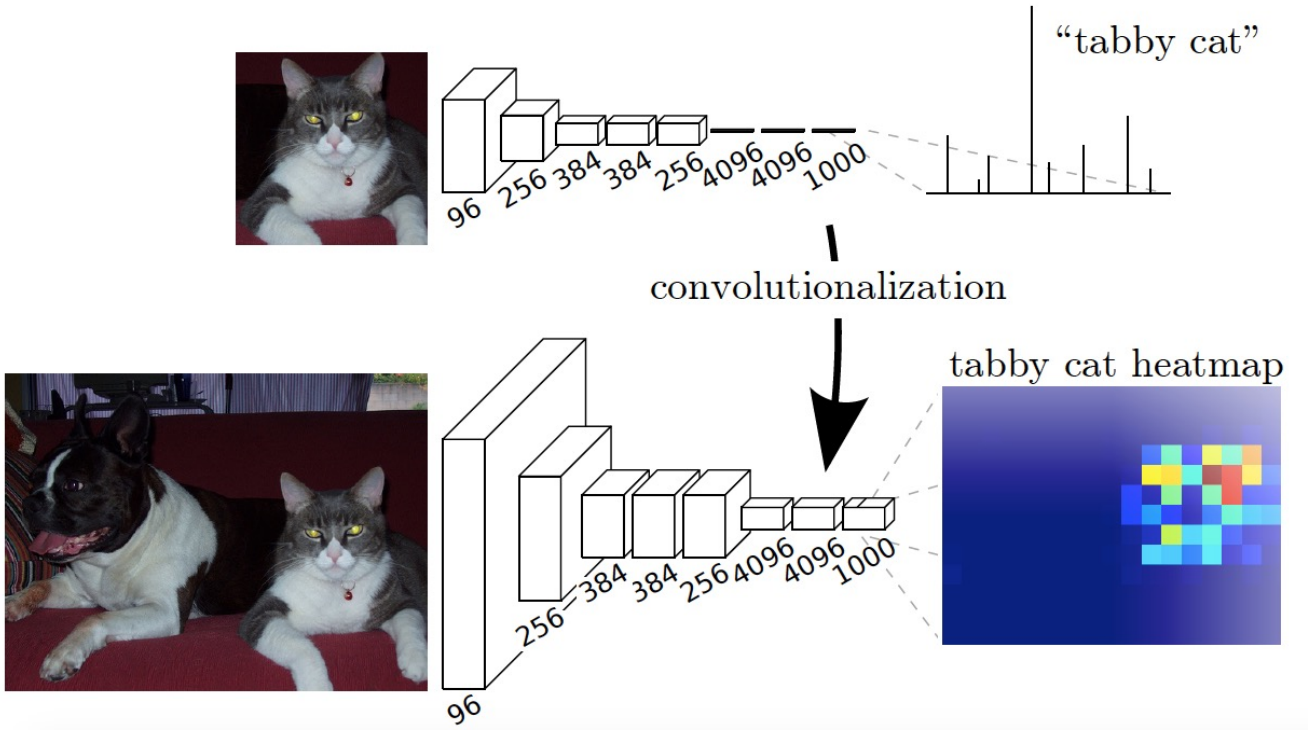
Early “Hacks”

- Do dense prediction as a post-process on top of an image classification CNN



Adapting a Classifier for Dense Labeling

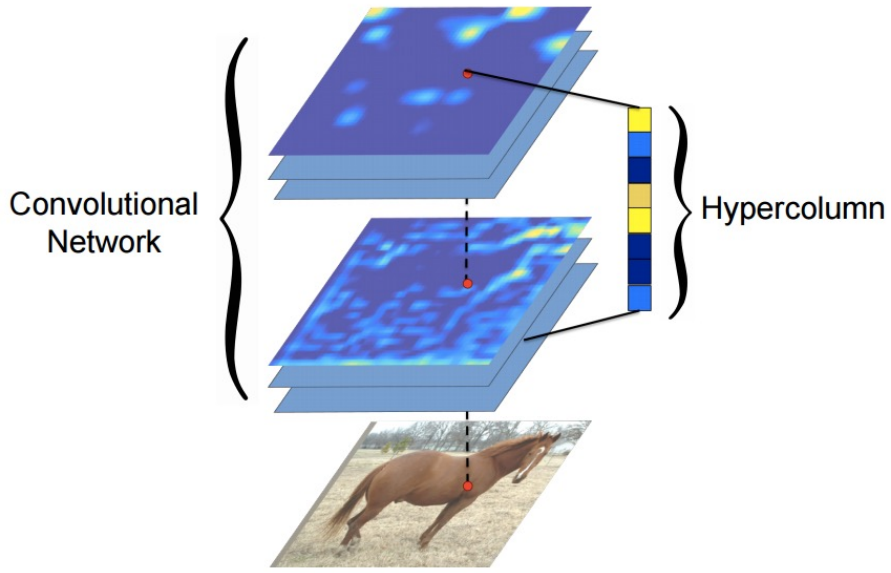
- Transforming fully connected layers in a network into convolution layers allows the classification model to output a spatial map



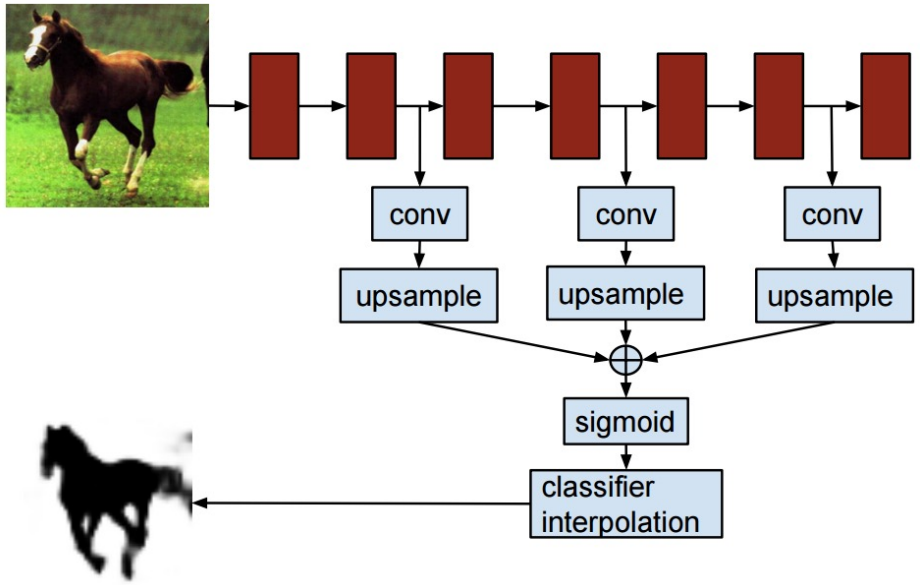
J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

Hypercolumns

- Idea: to obtain a feature representation for an individual pixel, upsample all feature maps to original image resolution and concatenate values from feature maps “above” that pixel



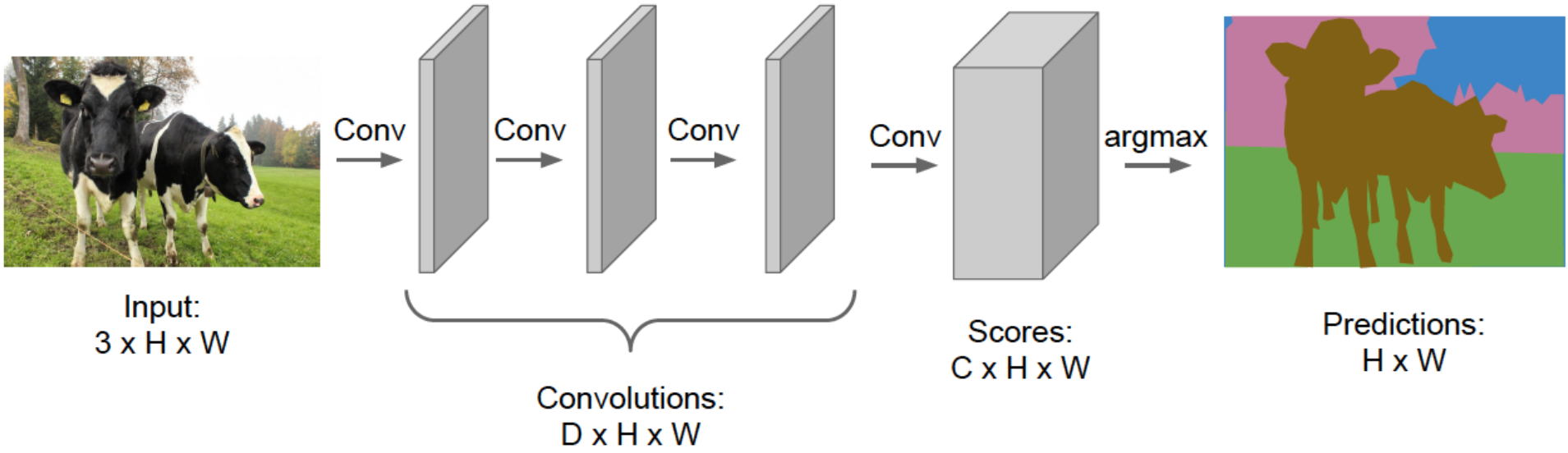
Representation as an end-to-end network:



B. Hariharan, P. Arbelaez, R. Girshick, and J. Malik, Hypercolumns for Object Segmentation and Fine-grained Localization, CVPR 2015

Fully Convolutional Networks (FCNs)

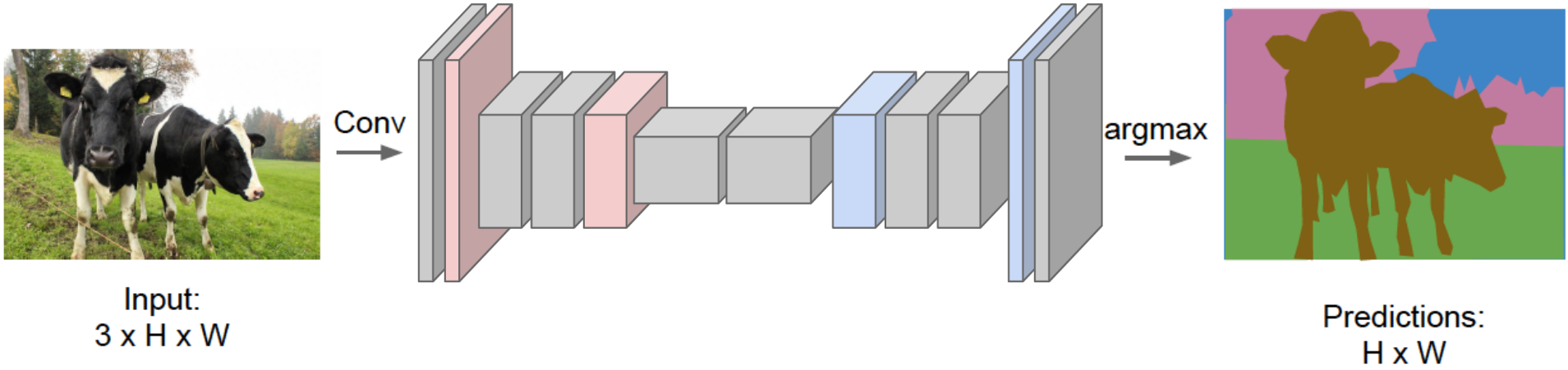
- Design a network with only convolutional layers, make predictions for all pixels at once
- Ideally, we want convolutions at full image resolution, but implementing that naively is too expensive and unnecessary



Source: Stanford CS231n

Fully Convolutional Networks (FCNs)

- Design a network with only convolutional layers, make predictions for all pixels at once
- Ideally, we want convolutions at full image resolution, but implementing that naively is too expensive and unnecessary
  - Solution: first downsample, then **upsample**



Source: Stanford CS231n

Upsampling in a Deep Network

- Review: regular convolution (stride 1, pad 0)

x_{11}	x_{12}	x_{13}	x_{14}
x_{21}	x_{22}	x_{23}	x_{24}
x_{31}	x_{32}	x_{33}	x_{34}
x_{41}	x_{42}	x_{43}	x_{44}

$*$

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

$=$

z_{11}	z_{12}
z_{21}	z_{22}

Upsampling in a Deep Network

- Review: regular convolution (stride 1, pad 0)

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix} * \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix}$$

- Matrix-vector form:

$$\begin{pmatrix} w_{11} & w_{12} & w_{13} & 0 & w_{21} & w_{22} & w_{23} & 0 & w_{31} & w_{32} & w_{33} & 0 & 0 & 0 & 0 & 0 \\ 0 & w_{11} & w_{12} & w_{13} & 0 & w_{21} & w_{22} & w_{23} & 0 & w_{31} & w_{32} & w_{33} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{11} & w_{12} & w_{13} & 0 & w_{21} & w_{22} & w_{23} & 0 & w_{31} & w_{32} & w_{33} & 0 \\ 0 & 0 & 0 & 0 & 0 & w_{11} & w_{12} & w_{13} & 0 & w_{21} & w_{22} & w_{23} & 0 & w_{31} & w_{32} & w_{33} \end{pmatrix} \begin{pmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ \vdots \\ x_{44} \end{pmatrix} = \begin{pmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{pmatrix}$$

4x4 input, 2x2 output

Upsampling in a Deep Network

- Transposed convolution (deconvolution?)

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} *^T \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}$$

$$\begin{pmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{pmatrix} \begin{pmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{pmatrix} = \begin{pmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{pmatrix}$$

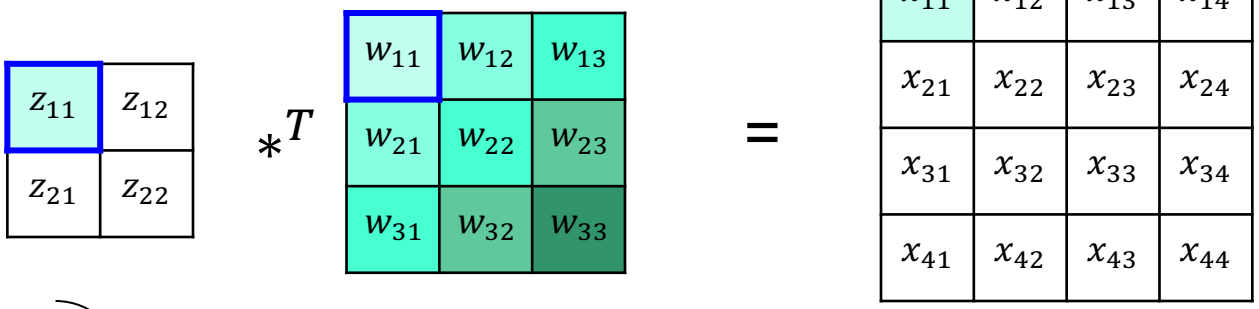
2x2 input, 4x4 output

Convolve input with transposed kernel matrix

Not an inverse of the original convolution operation, simply reverses dimension change!

Upsampling in a Deep Network

- Transposed convolution



$$\begin{pmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{pmatrix} \begin{pmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{pmatrix} = \begin{pmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{pmatrix}$$

$$x_{11} = w_{11} z_{11}$$

* Convolve input with *flipped* filter

Upsampling in a Deep Network

- Transposed convolution

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} *^T \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}$$

$$\begin{bmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{bmatrix} \begin{bmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{bmatrix} = \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{bmatrix}$$

$$x_{12} = w_{12}z_{11} + w_{11}z_{12}$$

Upsampling in a Deep Network

- Transposed convolution

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} *^{T} \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}$$

$$\begin{pmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{pmatrix} \begin{pmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{pmatrix} = \begin{pmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{pmatrix}$$

$$x_{13} = w_{13}z_{11} + w_{12}z_{12}$$

Upsampling in a Deep Network

- Transposed convolution

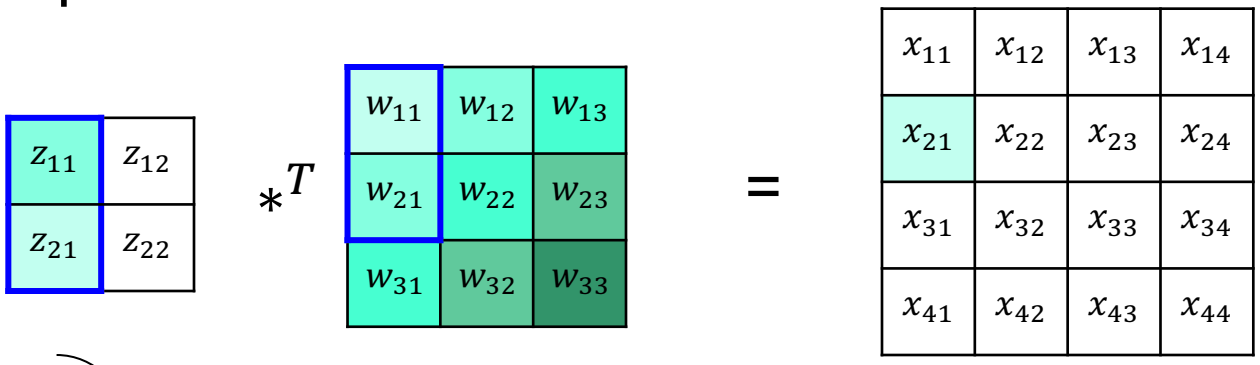
$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} *^T \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}$$

$$\begin{bmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{bmatrix} \begin{bmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{bmatrix} = \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{bmatrix}$$

$$x_{14} = w_{13} z_{12}$$

Upsampling in a Deep Network

- Transposed convolution



$$\begin{pmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{pmatrix} \begin{pmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{pmatrix} = \begin{pmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{pmatrix}$$

$$x_{21} = w_{21}z_{11} + w_{11}z_{21}$$

Upsampling in a Deep Network

- Transposed convolution

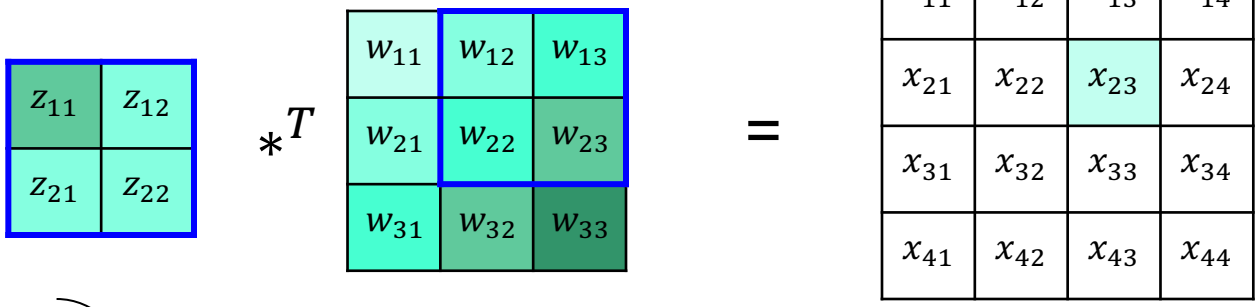
$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} *^T \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}$$

$$\begin{bmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{bmatrix} \begin{bmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{bmatrix} = \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{bmatrix}$$

$$x_{22} = w_{22}z_{11} + w_{21}z_{12} + w_{12}z_{21} + w_{11}z_{22}$$

Upsampling in a Deep Network

- Transposed convolution



$$\begin{pmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{pmatrix} \begin{pmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{pmatrix} = \begin{pmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{pmatrix}$$

$$x_{23} = w_{23}z_{11} + w_{22}z_{12} + w_{13}z_{21} + w_{12}z_{22}$$

Upsampling in a Deep Network

- Transposed convolution

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} *^T \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}$$

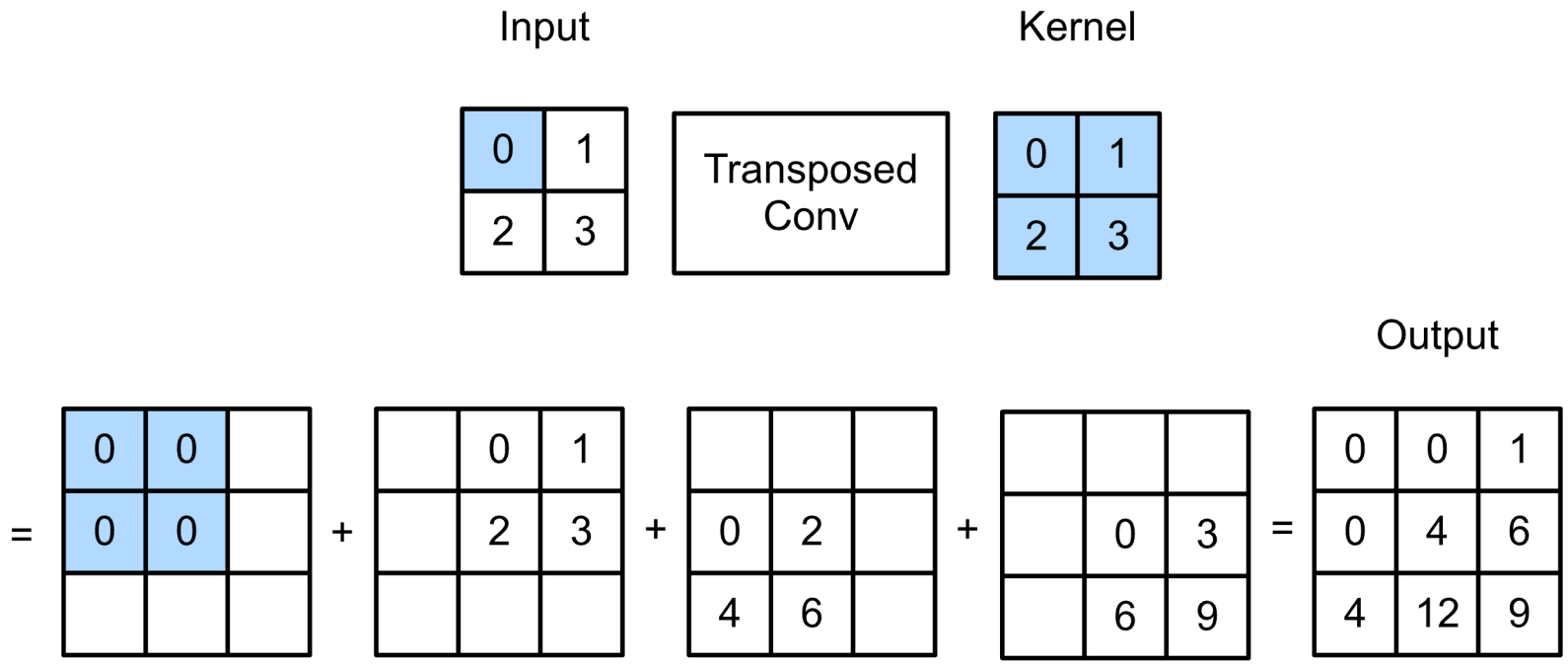
$$\begin{bmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ w_{13} & w_{12} & 0 & 0 \\ 0 & w_{13} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ w_{23} & w_{22} & w_{13} & w_{12} \\ 0 & w_{23} & 0 & w_{13} \\ w_{31} & 0 & w_{21} & 0 \\ w_{32} & w_{31} & w_{22} & w_{21} \\ w_{33} & w_{32} & w_{23} & w_{22} \\ 0 & w_{33} & 0 & w_{23} \\ 0 & 0 & w_{31} & 0 \\ 0 & 0 & w_{32} & w_{31} \\ 0 & 0 & w_{33} & w_{32} \\ 0 & 0 & 0 & w_{33} \end{bmatrix} \begin{bmatrix} z_{11} \\ z_{12} \\ z_{21} \\ z_{22} \end{bmatrix} = \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \\ x_{41} \\ x_{42} \\ x_{43} \\ x_{44} \end{bmatrix}$$

$$x_{23} = w_{23}z_{11} + w_{22}z_{12} + w_{13}z_{21} + w_{12}z_{22}$$

Think about bilinear interpolation

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution



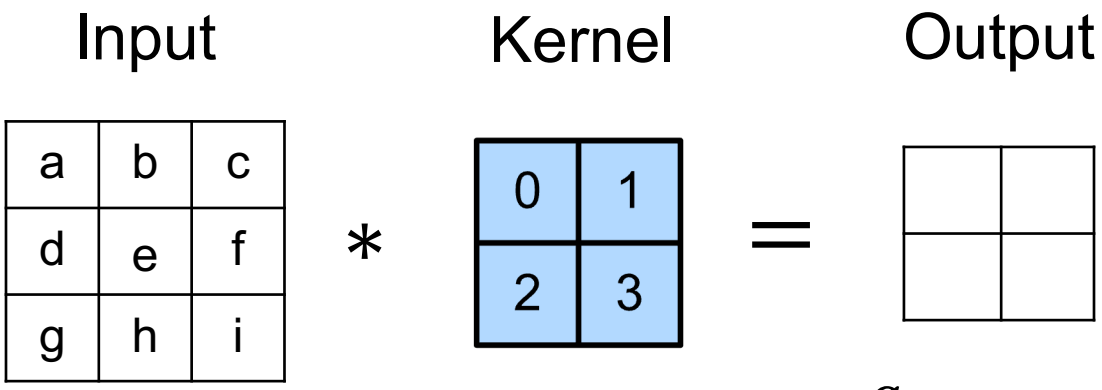
- Alternative interpretation: multiplying each input value with the shifted version of the kernel (assuming output stride = 1)

Image source: https://d2l.ai/chapter_computer-vision/transposed-conv.html

Additional Slides:
Example of Transposed Convolution

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution



Kernel

$$\begin{pmatrix} 0 & 1 & 0 & 2 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 2 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 2 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 2 & 3 \end{pmatrix}$$

Input

$$\begin{pmatrix} a \\ b \\ c \\ d \\ e \\ f \\ g \\ h \\ i \end{pmatrix} = \ldots$$

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

Input

0	1
2	3

$*^T$

Kernel

0	1
2	3

Kernel

$$\begin{pmatrix} 0 & 1 & 0 & 2 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 2 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 2 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 2 & 3 \end{pmatrix}^{T} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 4 \\ 6 \\ 4 \\ 12 \\ 9 \end{pmatrix}$$

Output

0	0	1
0	4	6
4	12	9

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 4 \\ 6 \\ 4 \\ 12 \\ 9 \end{pmatrix}$$

0 *

0	1	0
2	3	0
0	0	0

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 4 \\ 6 \\ 4 \\ 12 \\ 9 \end{pmatrix}$$

1 *

0	0	1
0	2	3
0	0	0

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 4 \\ 6 \\ 4 \\ 12 \\ 9 \end{pmatrix}$$

2 *

0	0	0
0	1	0
2	3	0

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 4 \\ 6 \\ 4 \\ 12 \\ 9 \end{pmatrix}$$

3 *

0	0	0
0	0	1
0	2	3

Upsampling w. Transposed Convolution

- An example with cross-correlation as convolution

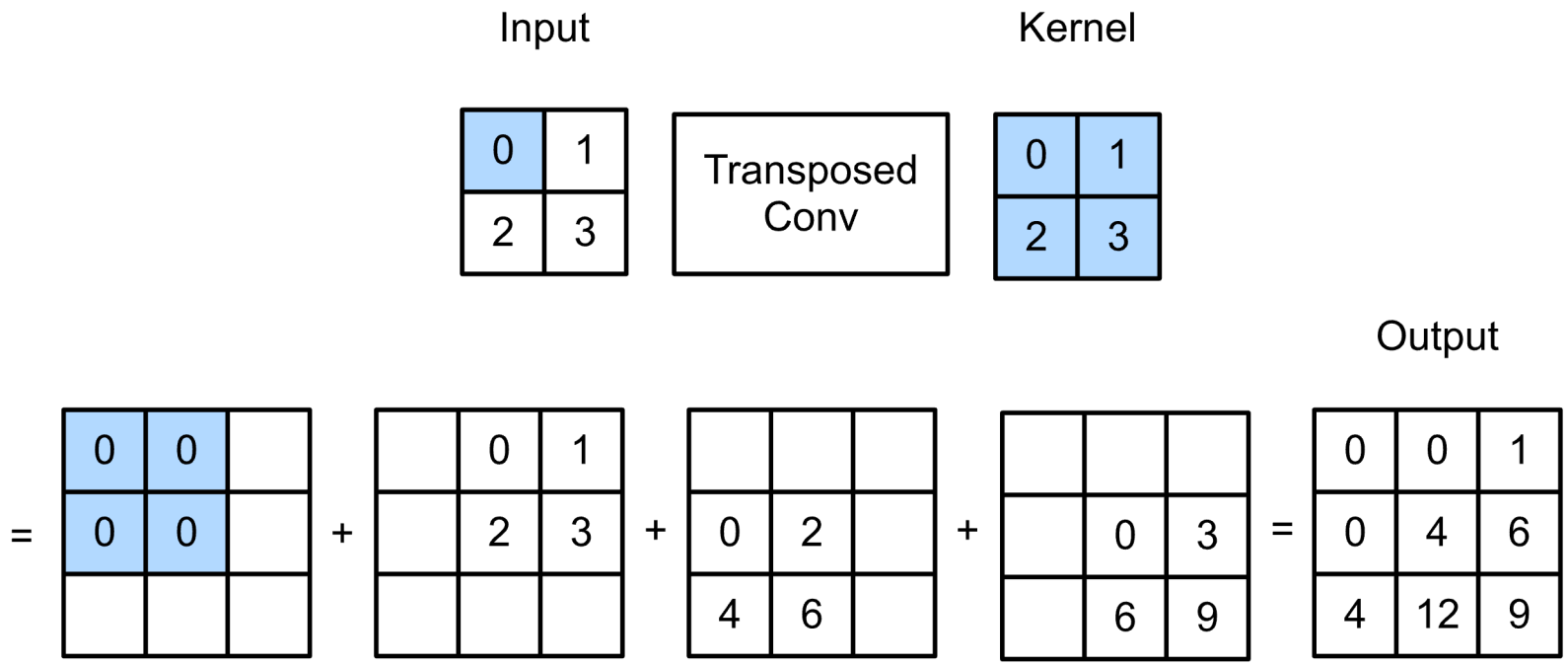
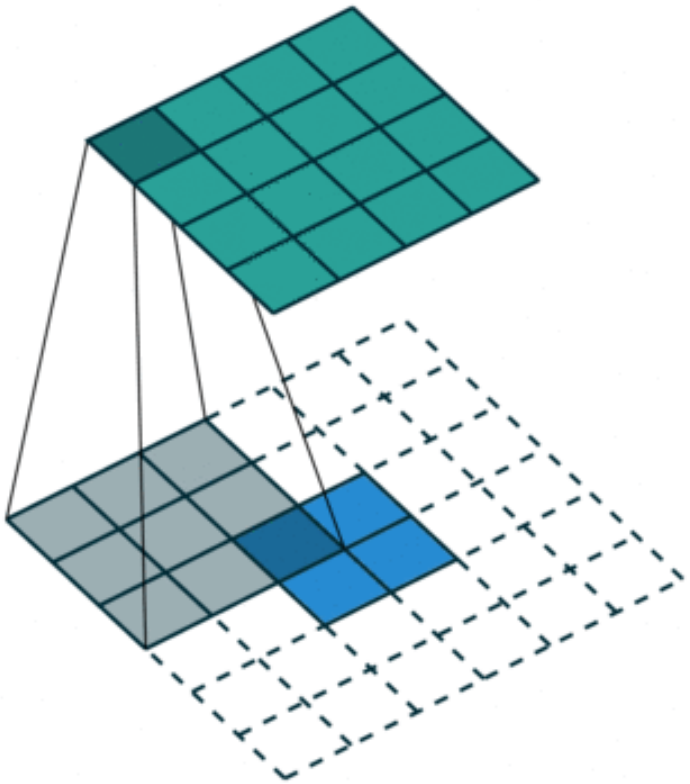


Image source: https://d2l.ai/chapter_computer-vision/transposed-conv.html

End of Additional Slides

Upsampling in a Deep Network

- Transposed convolution



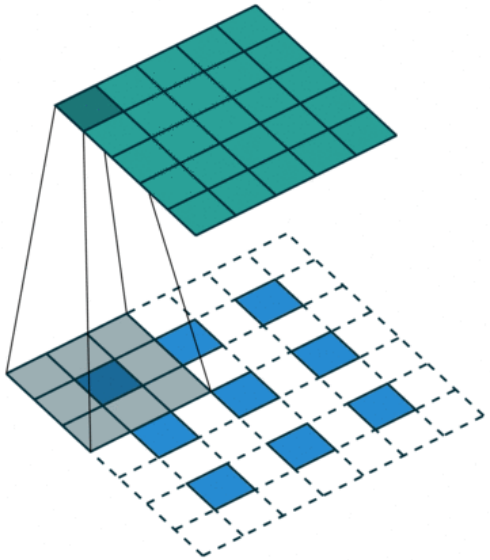
V. Dumoulin and F. Visin, A guide to convolution arithmetic for deep learning, arXiv 2018

Upsampling in a Deep Network

- *Transposed convolution*: to increase resolution, use *output stride* > 1
  - For stride 2, dilate the input by inserting rows and columns of zeros between adjacent entries
  - Sometimes called convolution with *fractional input stride* 1/2

output

input

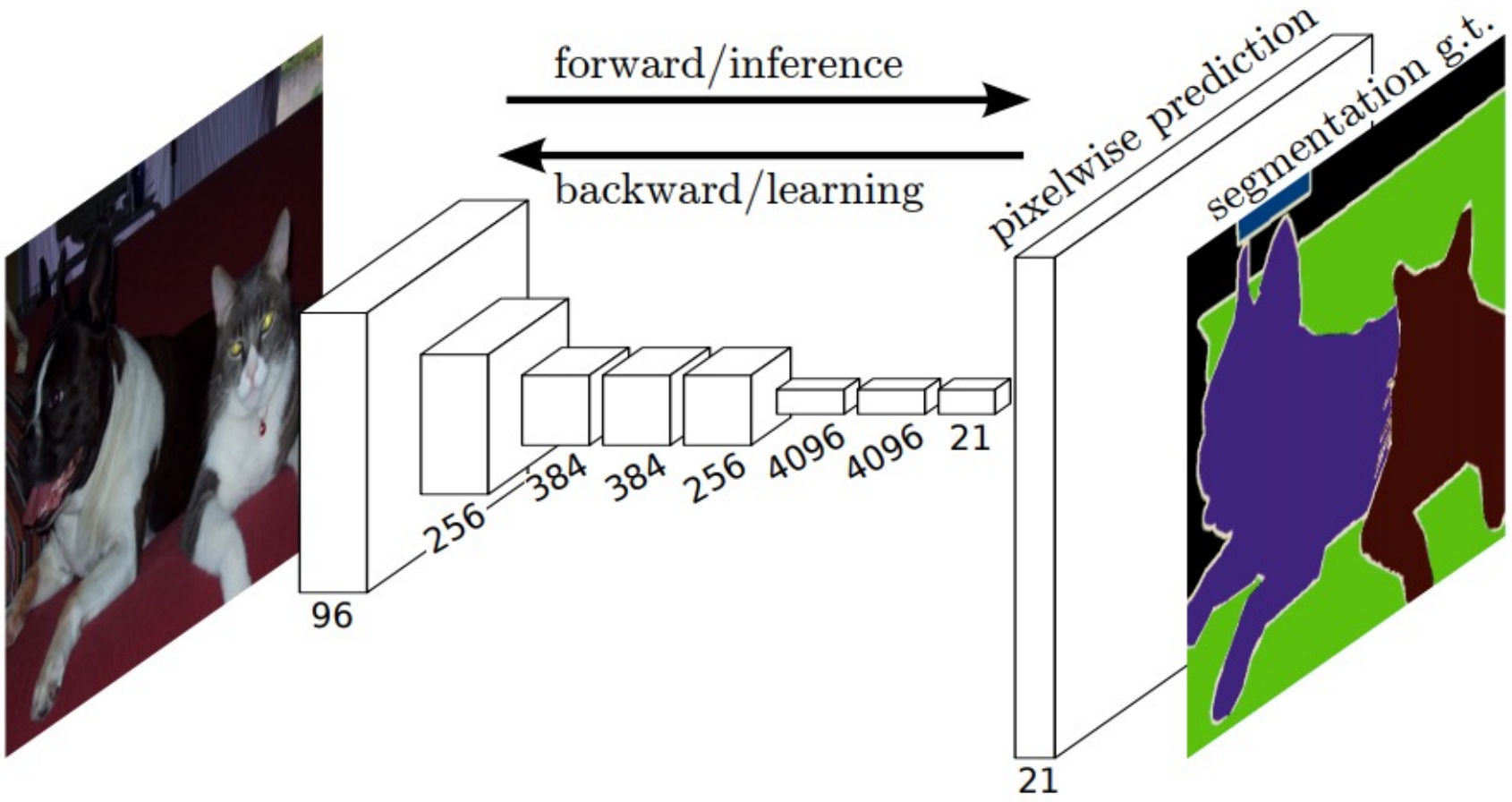


Q: What 3x3 filter would correspond to bilinear upsampling?

$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{4}$
$\frac{1}{2}$	1	$\frac{1}{2}$
$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{4}$

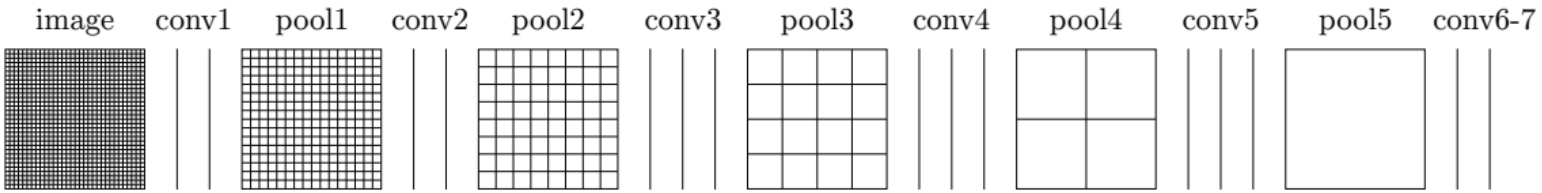
V. Dumoulin and F. Visin, A guide to convolution arithmetic for deep learning, arXiv 2018
See also https://distill.pub/2016/deconv-checkerboard/

Fully Convolutional Networks (FCNs)



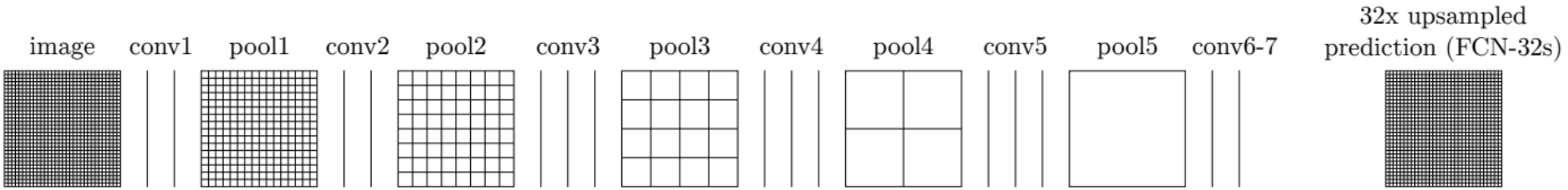
J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

Fully Convolutional Networks (FCNs)



J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

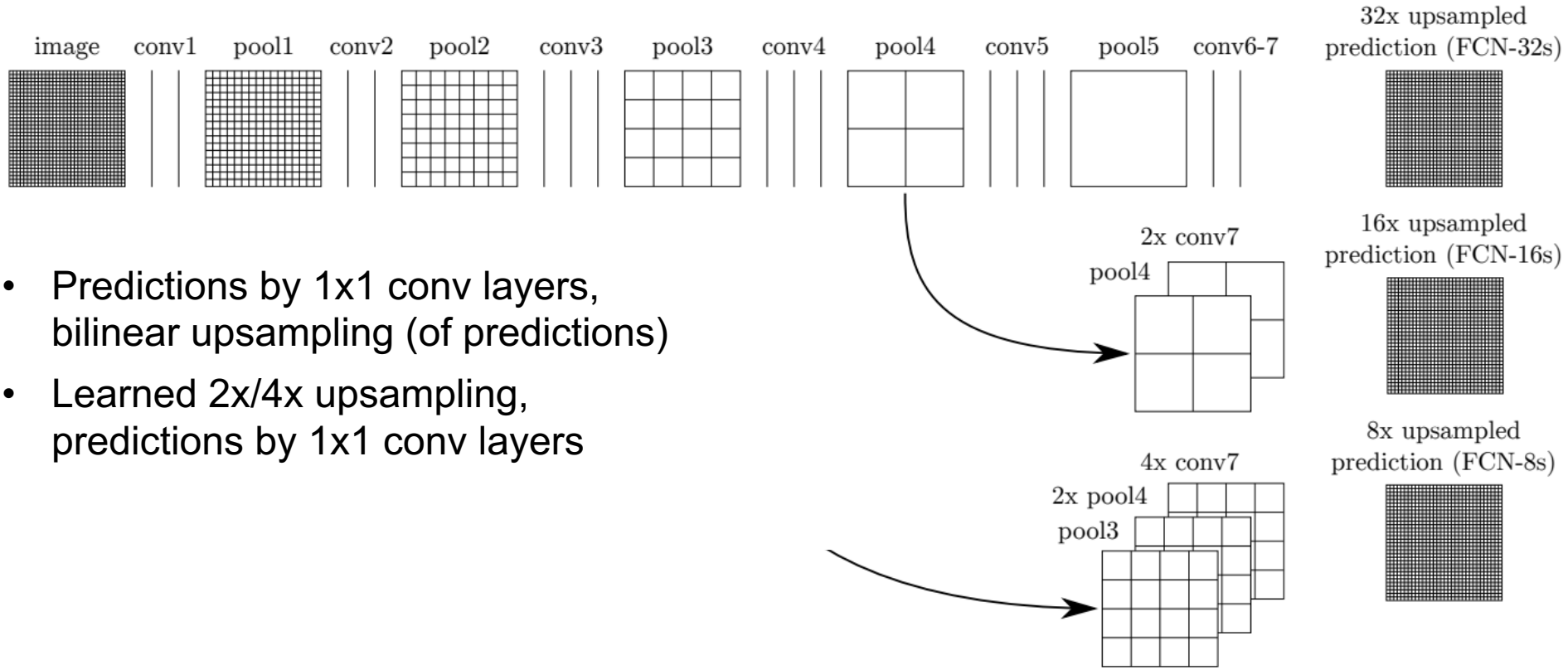
Fully Convolutional Networks (FCNs)



- Predictions by 1x1 conv layers,
  bilinear upsampling (of predictions)

J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

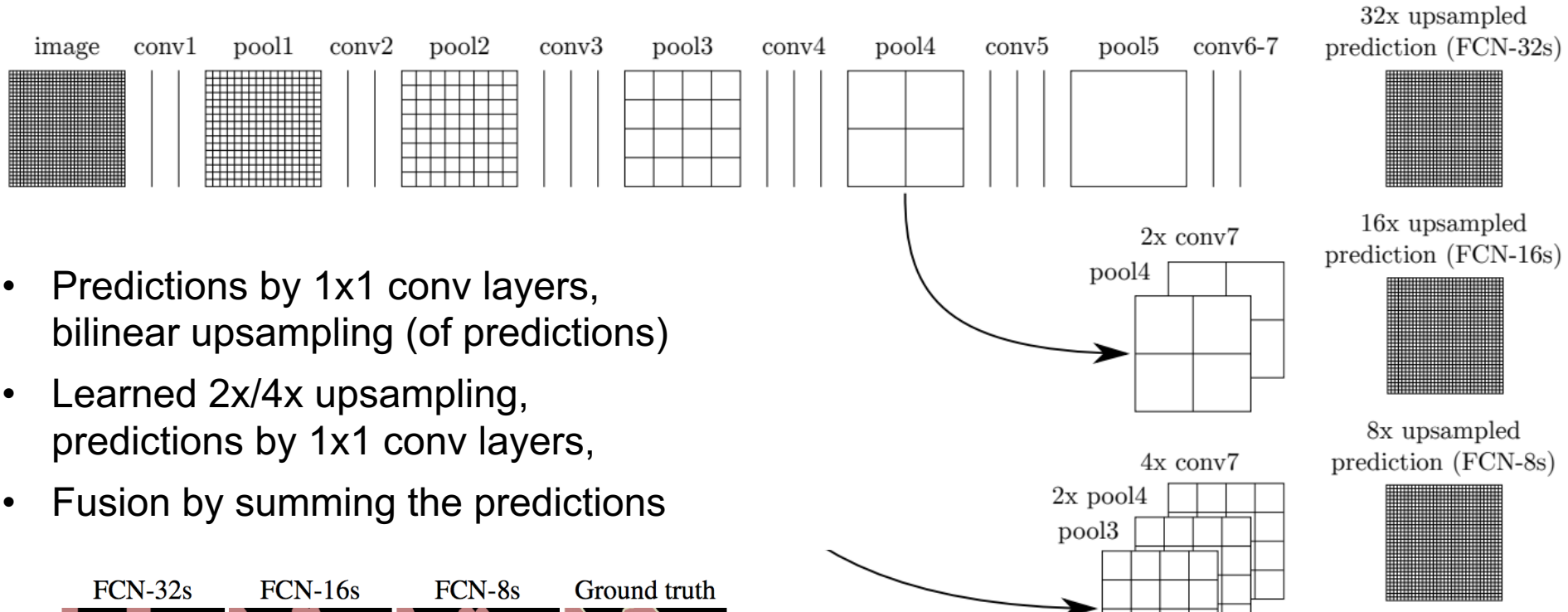
Fully Convolutional Networks (FCNs)



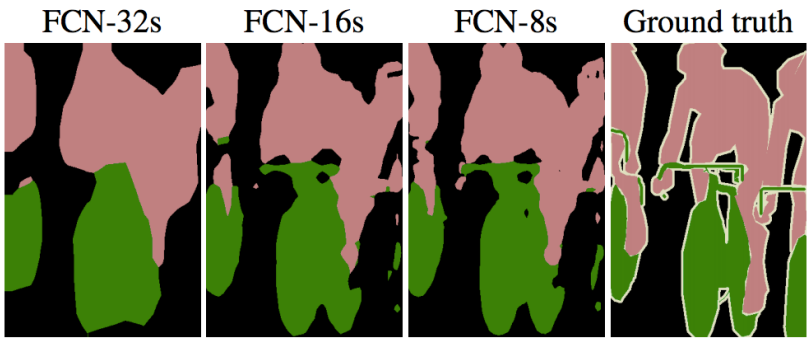
- Predictions by 1x1 conv layers, bilinear upsampling (of predictions)
- Learned 2x/4x upsampling, predictions by 1x1 conv layers

J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

Fully Convolutional Networks (FCNs)



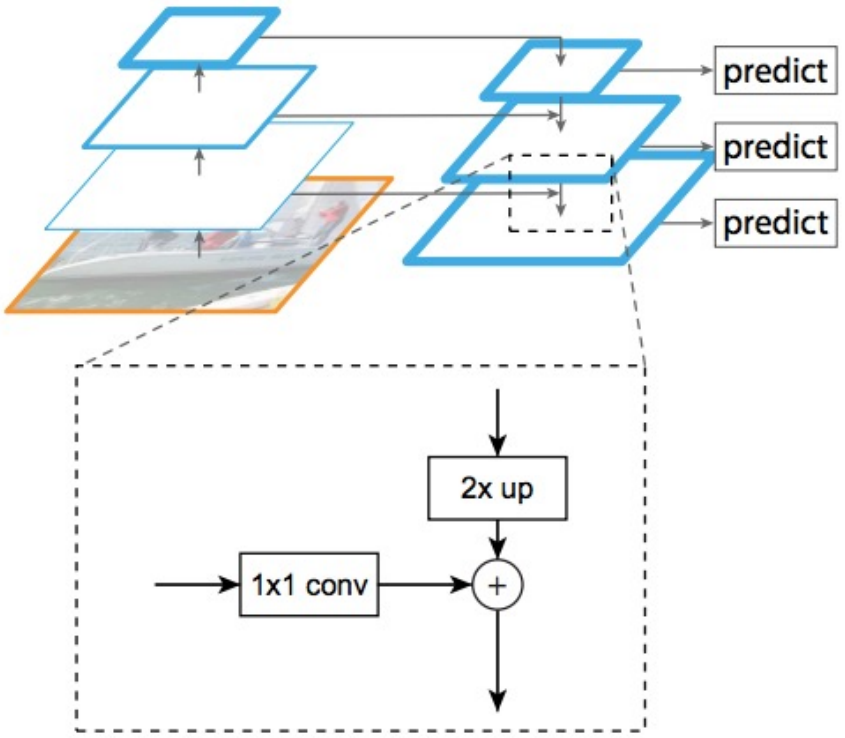
- Predictions by 1x1 conv layers, bilinear upsampling (of predictions)
- Learned 2x/4x upsampling, predictions by 1x1 conv layers,
- Fusion by summing the predictions



J. Long, E. Shelhamer, and T. Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

Recall: Feature pyramid networks

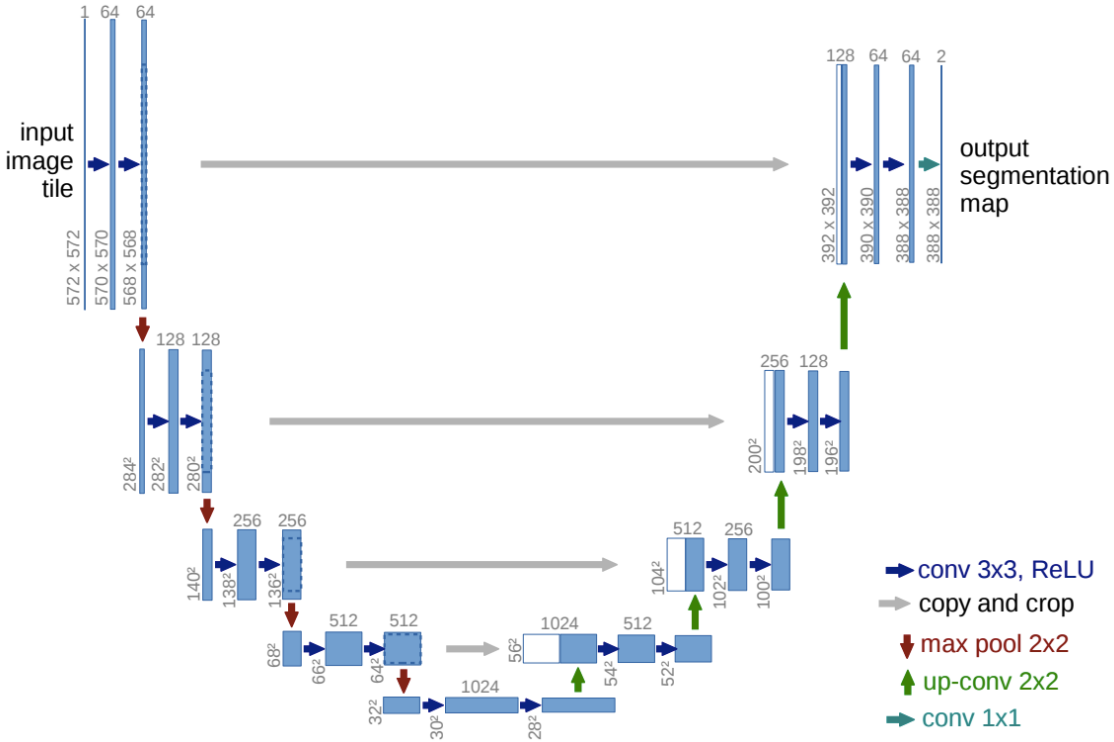
- Improve predictive power of lower-level feature maps by adding contextual information from higher-level feature maps
- Upsample from a higher level (lower resolution) and fuse to a lower level (higher resolution)
- FCN can be considered as a special case of FPN



T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, Feature pyramid networks for object detection, CVPR 2017.

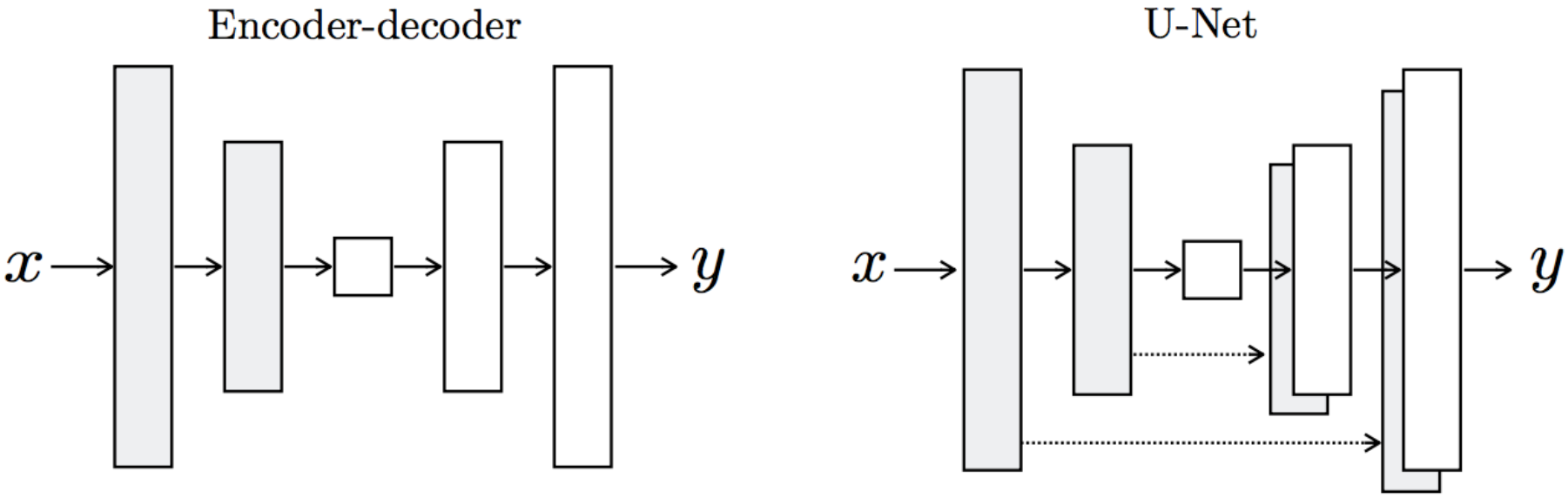
U-Net

- Like FCN, fuse upsampled higher-level feature maps with higher-res, lower-level feature maps
- Unlike FCN, fuse by concatenation, predict at the end



O. Ronneberger, P. Fischer, T. Brox U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015

U-Net: Upsampling

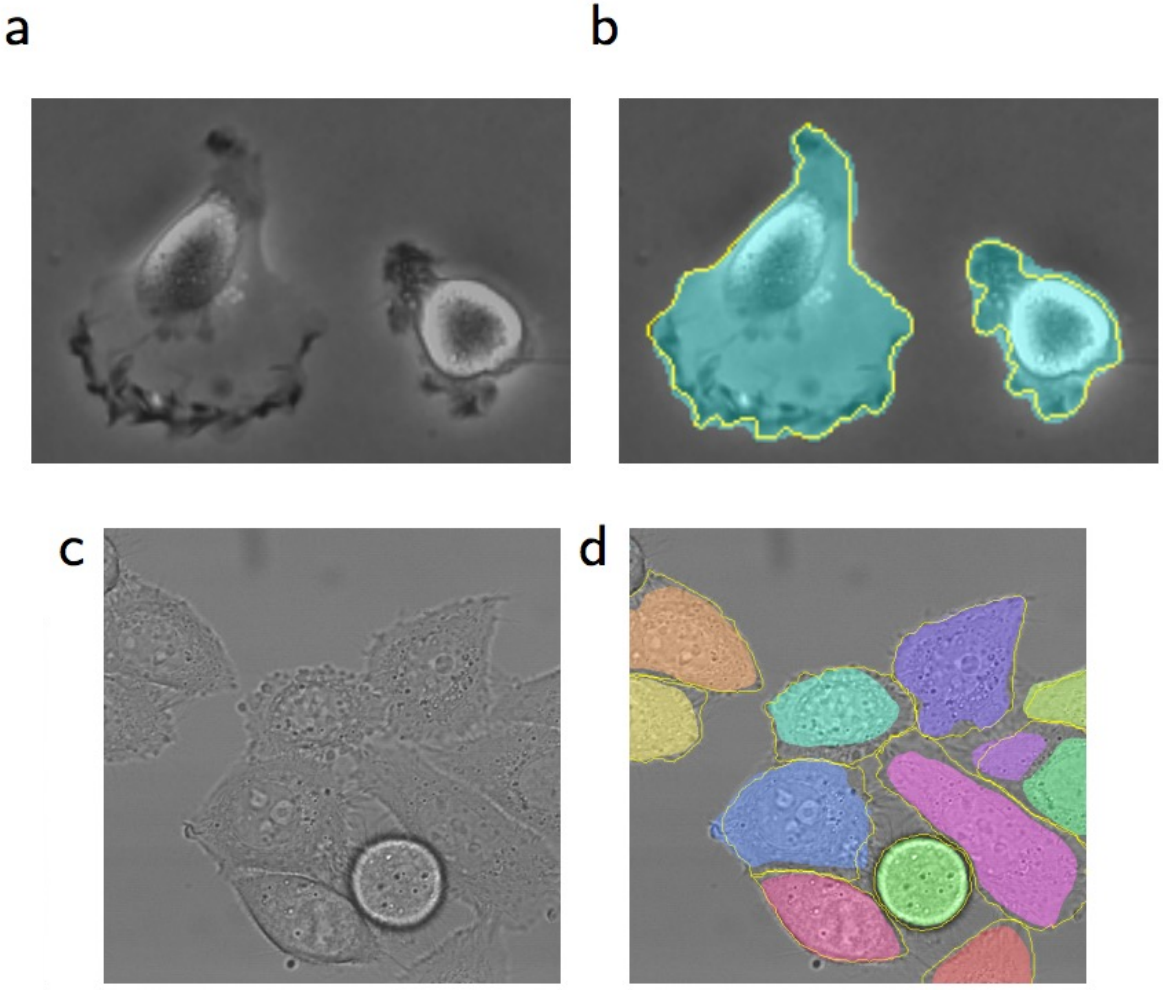


Key design:

- upsampling (from a previous level) using deconvolution
- fusing the upsampled features using concatenation

Figure source

U-Net: Results

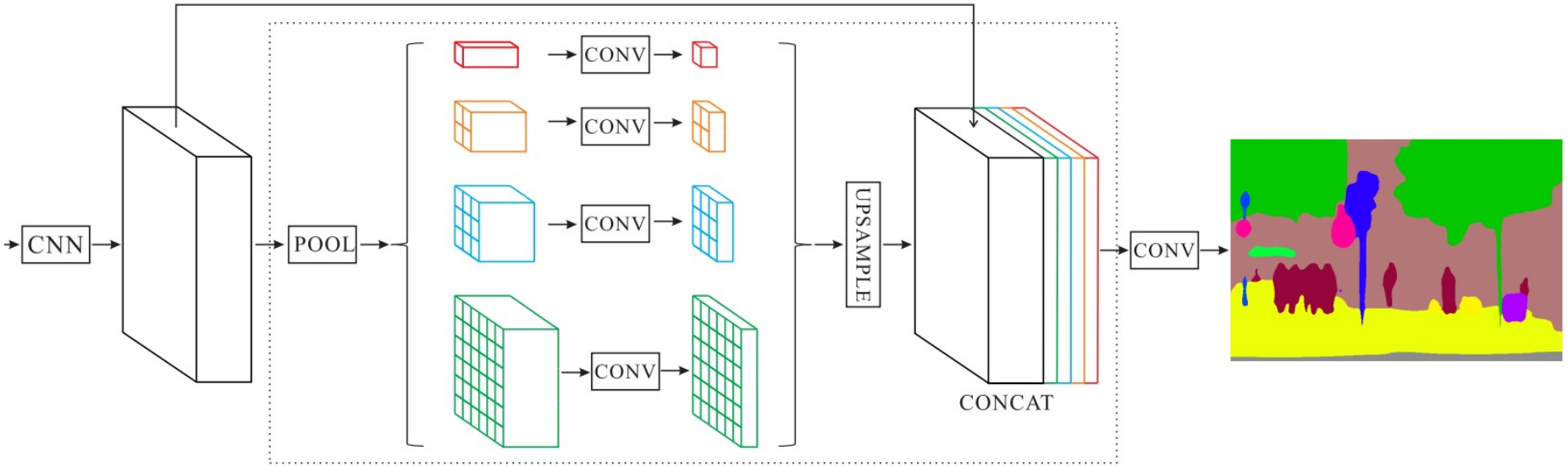


O. Ronneberger, P. Fischer, T. Brox U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015

Recap: Everything about Upsampling

- Transposed convolution (learnable bilinear upsampling) and FCN
- FPN and U-Net

Incorporate Global Contextual Cues



- Global average pooling using different sizes (e.g, 2x2, 4x4)
- Upsample the pooled features and concatenate them

H. Zhao, J. Shi, X. Qi, X. Wang, J. Jia, Pyramid Scene Parsing Network, CVPR 2017

PSPNet: Results



(a) Image (b) Ground Truth FCN-8

PSPNet: Results



Recall: Instance Segmentation

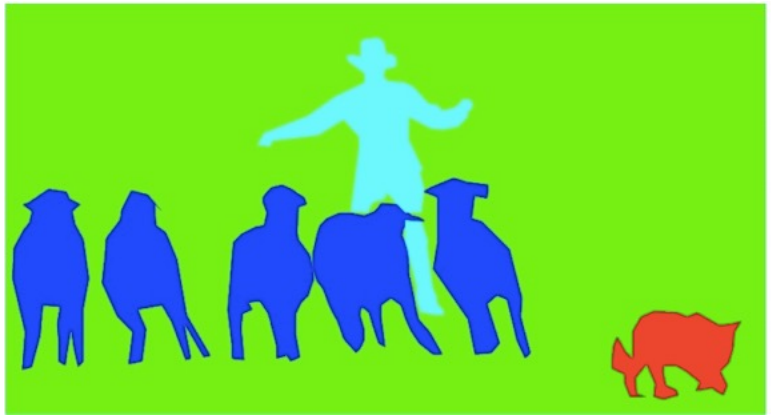




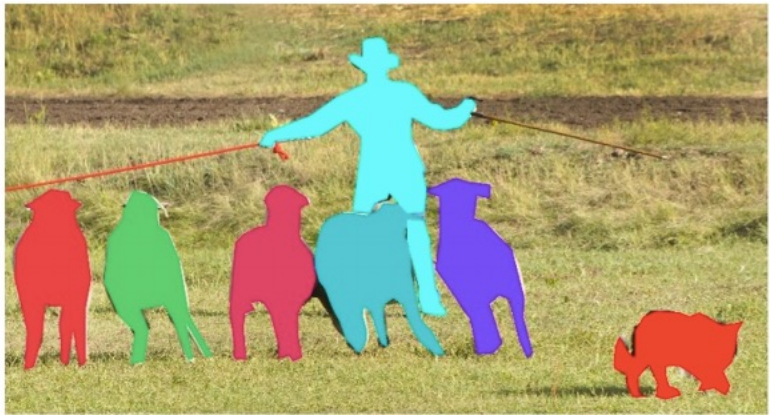




Semantic vs. Instance Segmentation



semantic segmentation



instance segmentation

- Stuff: defined by a homogenous or repetitive patterns of fine-scale properties, but as no specific or distinct spatial extent or shape
- Object: must have a specific size and shape

Forsyth, D. A., Malik, J., Fleck, M. M., Greenspan, H., Leung, T., Belongie, S., ... & Bregler, C. Finding pictures of objects in large collections of images. *International workshop on object representation in computer vision,* 1996.

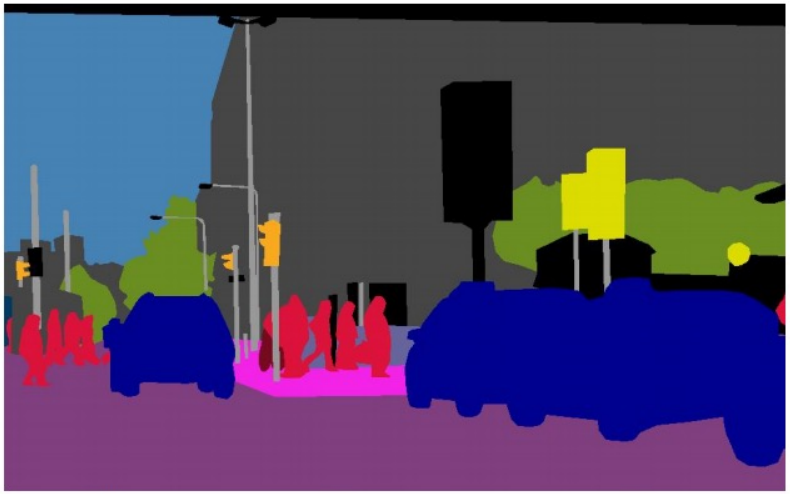
Semantic vs Instance Segmentation

Instance Segmentation	Semantic Segmentation
Only for Objects (shape & size)	Objects & Stuff (color & texture)
Sparse Labeling	Dense Labeling
Instance Separation	Pixel Prediction

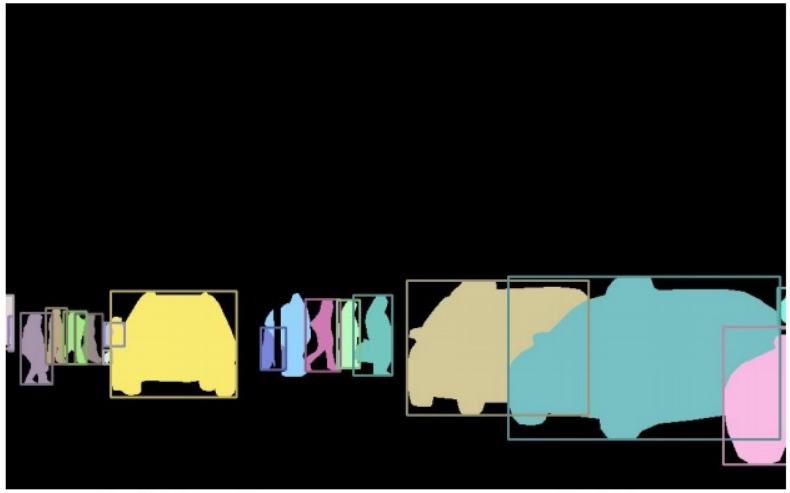
Scene Parsing



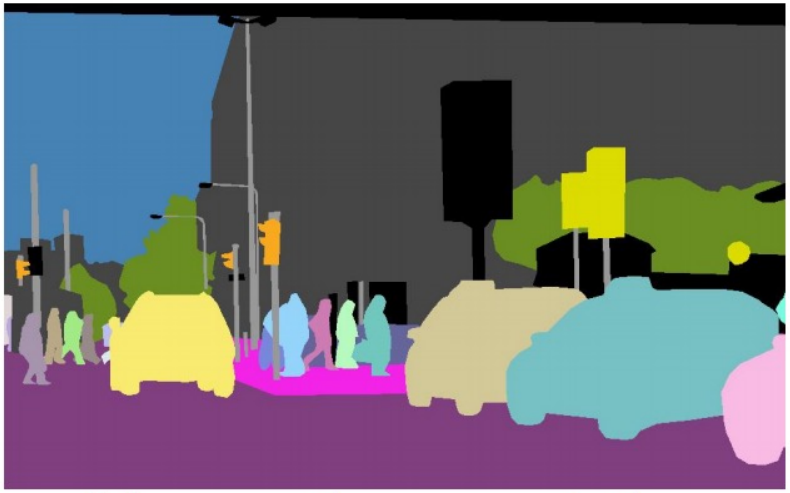
(a) image



(b) semantic segmentation



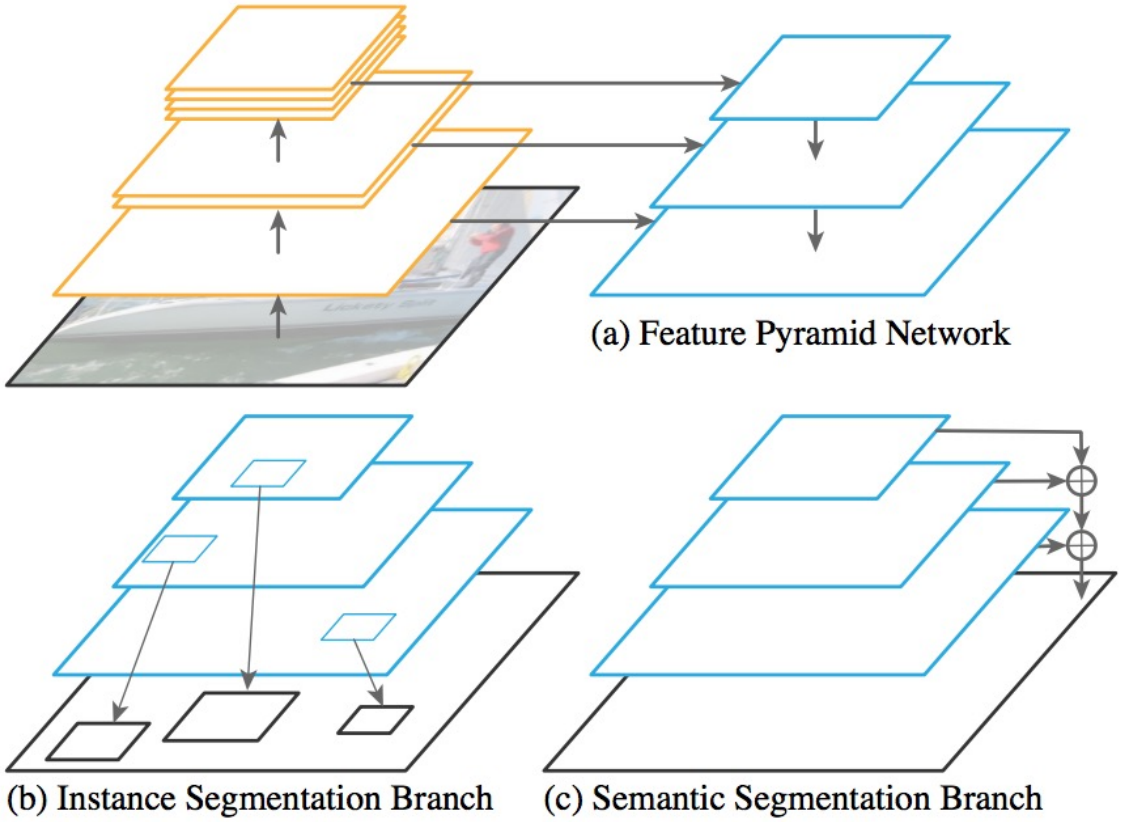
(c) instance segmentation



(d) panoptic segmentation

A. Kirillov, K. He, R. Girshick, C. Rother, P. Dollar, Panoptic Segmentation, arXiv 2018

Early Hacks: Panoptic FPNs



Using FPN as the backbone

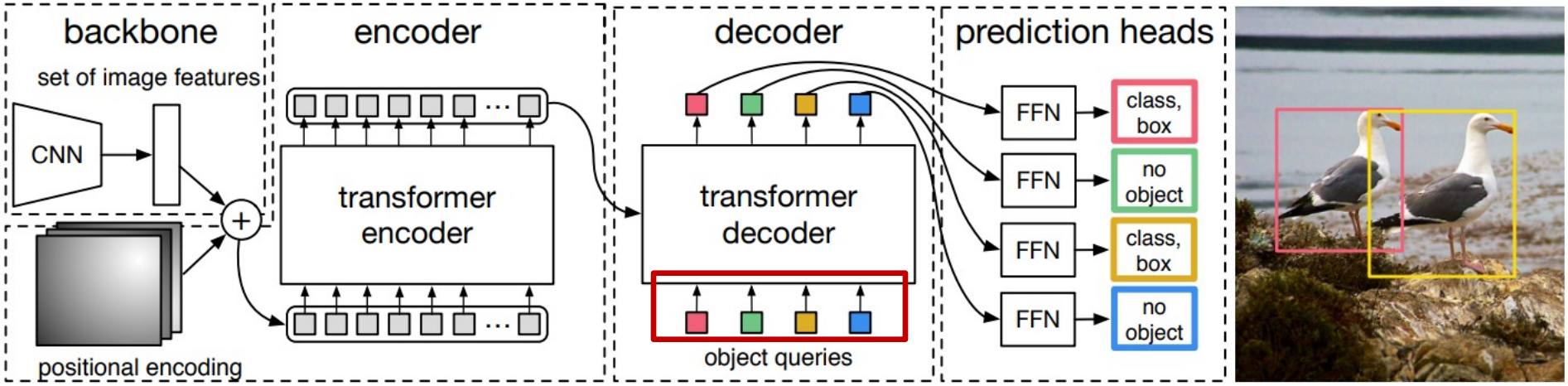
+ Mask RCNN for instance segmentation

+ FCN for semantic segmentation (pixel labeling)

A. Kirillov, R. Girshick, K. He, P. Dollar, Panoptic Feature Pyramid Networks, CVPR 2019

Review: A Transformer-based Detector (DETR)

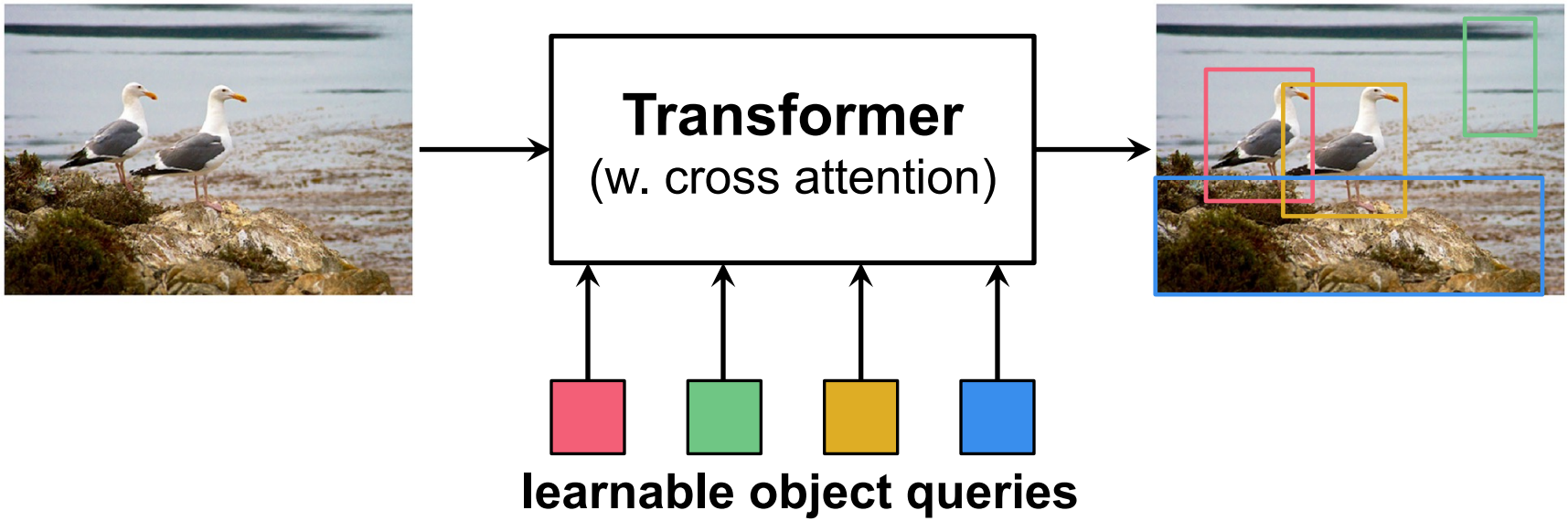
- Learn a set of “object queries” (may not have geometric meanings) from data
- Decode an object (category / box / score) from each “object query”
- Training involves a bipartite matching between prediction and ground-truth boxes



Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In *ECCV*.

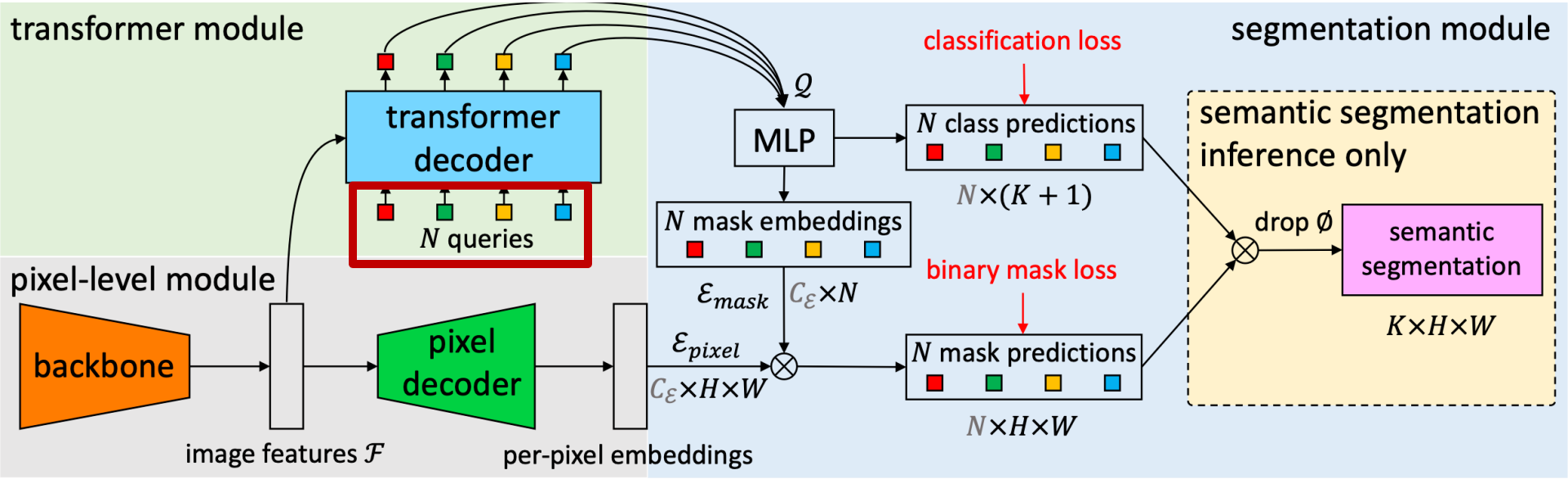
Review: A Transformer-based Detector (DETR)

- Learn a set of "object queries" (may not have geometric meanings) from data
- Decode an object (category / box / score) from each "object query"
- **What if we ask the model to decode a mask per query?**



Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In *ECCV*.

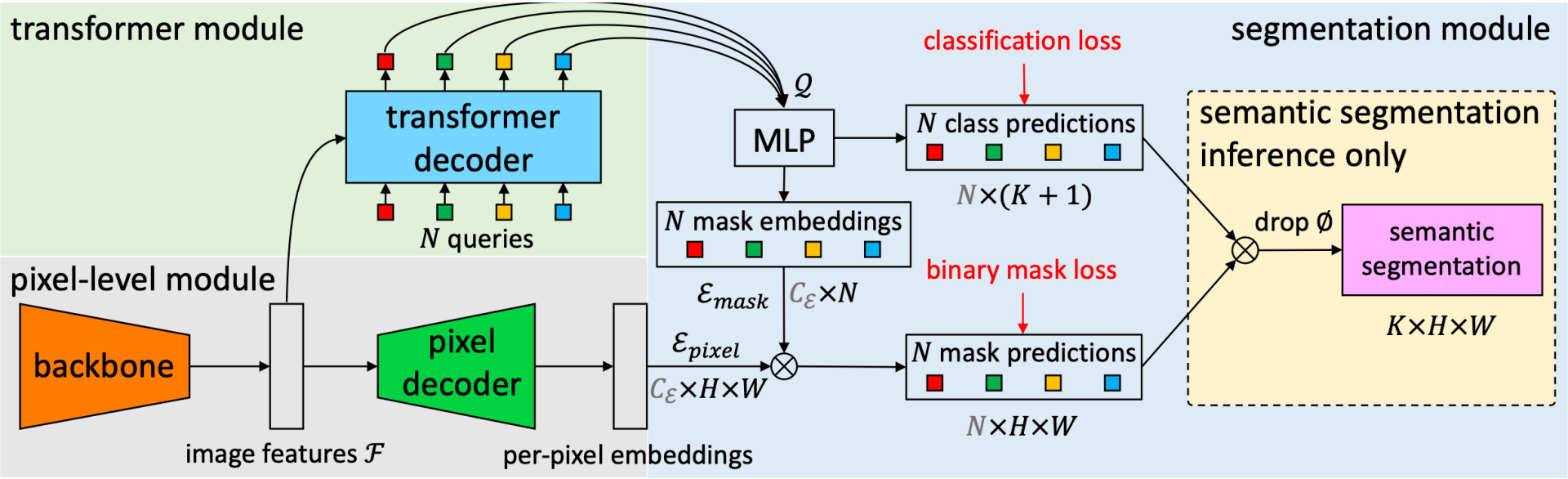
Beyond Pixel Labeling: MaskFormer



- Consider *N* learned "mask queries"
- Predict *N* mask embeddings (w. classification loss)
- Use cross-attention between the mask embeddings and the 2D feature map to decode the masks

Cheng, B., Schwing, A., & Kirillov, A.
Per-pixel classification is not all you need for semantic segmentation. *NeurIPS* 2021.

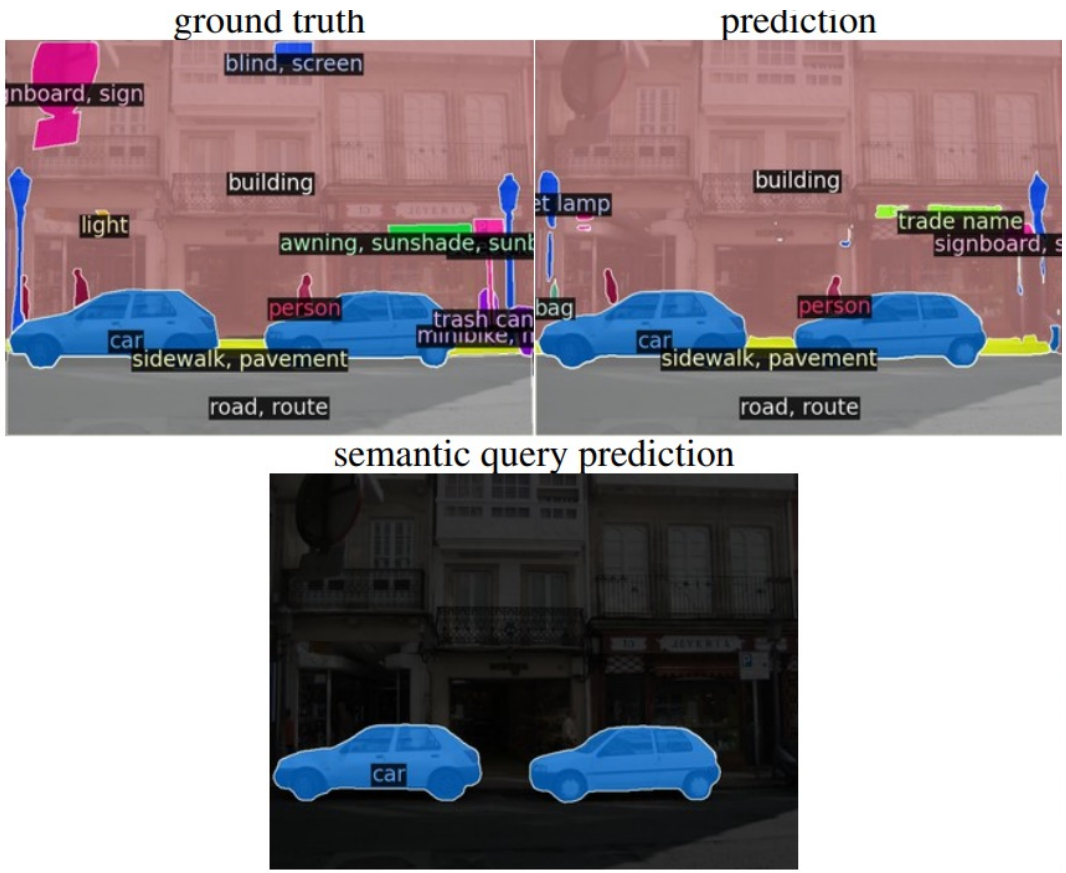
Beyond Pixel Labeling: MaskFormer



- Training: matching between predicted masks and ground-truth masks

Cheng, B., Schwing, A., & Kirillov, A.
Per-pixel classification is not all you need for semantic segmentation. *NeurIPS* 2021.

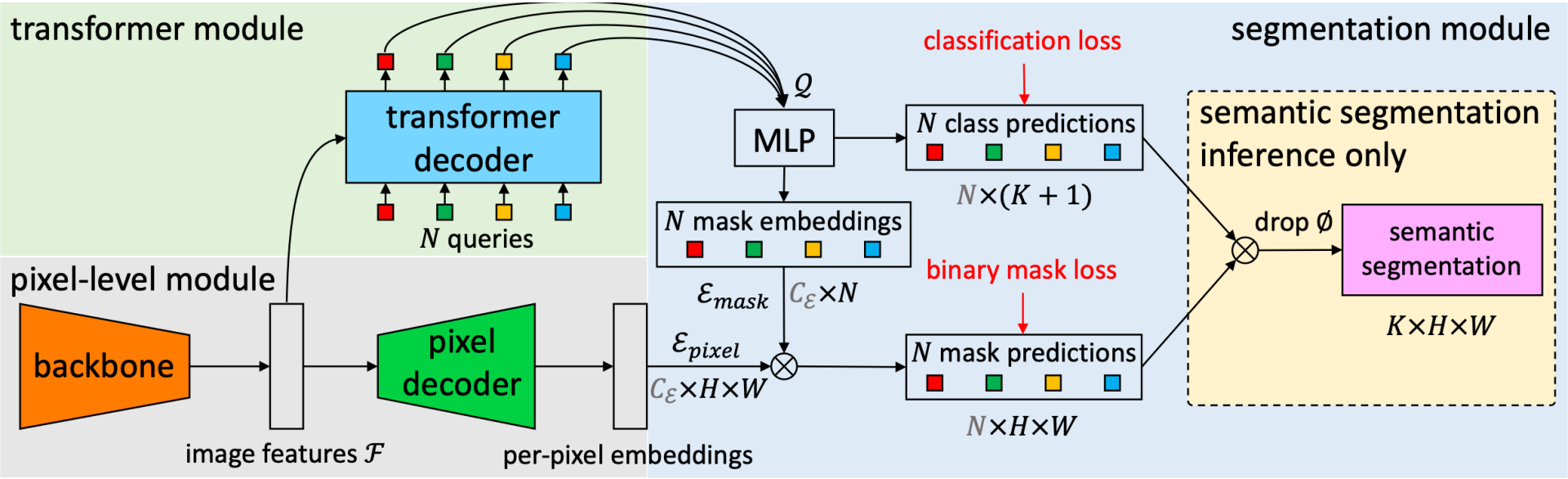
Beyond Pixel Labeling: MaskFormer



- Set *N* = *#categories* and use bipartite matching, so that each query predicts the masks for one category

Cheng, B., Schwing, A., & Kirillov, A.
Per-pixel classification is not all you need for semantic segmentation. *NeurIPS* 2021.

MaskFormer for Scene Parsing



- Can be extended to handle scene parsing
- Learning mask queries for both objects and stuff
  - Each query -> object instance / stuff mask

Cheng, B., Misra, I., Schwing, A. G., Kirillov, A., & Girdhar, R. (2022). Masked-attention mask transformer for universal image segmentation. In *CVPR*

Dense Labeling: Summary

- Semantic segmentation
  - Fully convolutional networks
  - Contextual cues for semantic segmentation
  - Transformer-based models
- Scene parsing
  - Semantic segmentation (stuff)
    + instance segmentation (object)
  - Transformer-based models

Pose Estimation: 2D Keypoints

- Given K keypoints, train model to predict K m x m *one-hot* maps



K. He, G. Gkioxari, P. Dollar, and R. Girshick, Mask R-CNN,
ICCV 2017 (Best Paper Award)

Pose Estimation: 3D Surfaces

- Predict dense key points on a parametric surface (human body model)



R.A. Guler, N. Neverova, I. Kokkinos,
DensePose: Dense Human Pose Estimation In The Wild, CVPR 2018